Sentiment Analysis of IMDB Movie Reviews

From a NumPy-only n-gram baseline to fine-tuned transformers and a LoRA-adapted large language model

CDS550 Programming with Generative AI — Individual Project Report

Submitted: 1 November 2026 (due 3 November 2026)

Summary. The task is to label 25,000 English IMDB movie reviews as positive or negative; Kaggle scores accuracy. The project went through seven stages, each motivated by an analysis of the previous one. Stage 1 implements a transparent baseline from scratch in NumPy: n-gram features in a custom sparse matrix, and naive Bayes and logistic regression with a hand-written L-BFGS optimiser. The best of 42 configurations, NB-LR with word 1–3-grams, reaches 91.78% in 5-fold cross-validation and 0.91730 on the public leaderboard. Its errors (contrast, sarcasm, mixed opinions) motivated Stage 2, which fine-tunes the pre-trained transformer DeBERTa-v3-large: 96.72% on a 2,500-review hold-out, 0.96880 public, and 0.97050 for an average of four transformer runs and NB-LR. Stage 3 replaces the small hold-out by 5-fold cross-validation, which gives an out-of-fold (OOF) prediction for every training review. DeBERTa reaches 96.73% OOF and the average of its five fold models 0.97150 public; ELECTRA-large, self-training and a re-run add nothing. All encoders make nearly the same errors (score correlation 0.98–0.99), so Stage 4 fine-tunes a far larger, differently pre-trained model: the 8-billion-parameter language model Qwen3-8B-Base, adapted with LoRA (0.57% of its weights trained). On the same folds it reaches 97.02% OOF, a significant gain over DeBERTa (298 against 224 discordant reviews, exact McNemar p = 0.0014), mostly on long reviews. Combining it with DeBERTa adds at most 0.05 points, below a threshold fixed in advance. Copying the training label to the 227 test reviews that duplicate a training review changes 4 predictions. Stage 5 rents a 96 GB cloud GPU to train the 14-billion-parameter Qwen3-14B-Base with the identical recipe: 97.15% OOF, 0.13 points above the 8B model (p = 0.07). The averaged scores of DeBERTa and both Qwen3 models reach 97.20% OOF, above the threshold fixed before the run. Stage 6 trains the 24-billion-parameter Mistral-Small-24B-Base from a different developer: 97.25% OOF, the best single model but not significantly better than Qwen3-14B (p = 0.21). The average of all four models reaches 97.30% and replaces the Stage 5 average by a pre-registered rule. Stage 7 adds Qwen3.8-27B, a post-trained model with a hybrid linear-attention architecture: 97.30% OOF, level with Mistral-24B (p = 0.54). The five-model average reaches 97.344%, below the pre-registered replacement threshold of 97.384%. The two final submissions are the four-model average (D) and the five-model average (E), both with the backfill: 0.97620 and 0.97650 on the public leaderboard. E replaced the Stage 4 stack, which the rule would have kept; Section 6.11 explains this choice. Most of the remaining 3% of errors are reviews whose text contradicts their star-rating label.

1. Introduction and Motivation

Sentiment analysis — deciding whether a piece of text expresses a favourable or unfavourable opinion (Pang & Lee, 2008) — is one of the most widely deployed natural-language-processing tasks. Studios and streaming platforms use it to aggregate audience reaction, retailers monitor product feedback with it, and it is a standard first benchmark for any new text-classification method. Movie reviews are a particularly instructive domain: reviewers mix plot summary with opinion, often use irony ("so bad it is good"), and frequently rate several aspects of a film (acting, script, soundtrack) differently in one text.

This project uses the Large Movie Review Dataset (IMDB) introduced by Maas et al. (2011), a benchmark of 50,000 highly polar reviews. The class competition re-shuffles the data into 25,000 labelled training reviews and 25,000 unlabelled test reviews; the public leaderboard scores 40% of the test set and the final ranking uses the remaining 60%.

My motivation was twofold. First, to understand the classical methods deeply by implementing them myself — all feature extraction, the sparse-matrix arithmetic, naive Bayes, logistic regression and the L-BFGS optimiser are written in plain NumPy, with no scikit-learn or deep-learning library — and to see how far carefully engineered n-gram models can go. Second, to measure what modern pre-trained models add on top of this baseline and, using each stage's error analysis, to explain where the gain comes from. The report follows the seven stages in order. Stage 1 (Sections 4.1–4.3, 5.1–5.3, 6.1–6.2) establishes a fully reproducible, interpretable reference. Stage 2 (Sections 4.4–4.6, 5.4, 6.3–6.4) fine-tunes DeBERTa-v3-large and RoBERTa-large and combines them with the baseline. Stage 3 (Sections 4.7, 5.5, 6.5) replaces the single hold-out by 5-fold cross-validation, so that every training review is used for both training and evaluation. Stage 4 (Sections 4.8–4.10, 5.5, 6.6–6.7) fine-tunes the large language model Qwen3-8B-Base with LoRA and combines it with DeBERTa. Stage 5 (Sections 4.11, 5.6, 6.8) repeats the Stage 4 recipe with the larger Qwen3-14B-Base on a rented cloud GPU. Stage 6 (Sections 4.12, 5.7, 6.9) applies the same recipe to Mistral-Small-24B-Base, a model from a different developer. Stage 7 (Sections 4.13, 5.8, 6.10) applies it to Qwen3.8-27B, a newer model with a different architecture that is only released in post-trained form. Section 6.11 reports every Kaggle submission and the final selection, and Section 7 discusses the remaining errors.

2. Problem Description

Given a review text x, predict a label y ∈ {negative, positive}. Formally, the training set is D = {(x_i, y_i)}, i = 1 … 25,000, and the goal is a function f that maximises classification accuracy on the 25,000 held-out test reviews:

$$Accuracy = (1/N)\ \Sigma_i\ \mathbb{1}[f(x_i) = y_i], \quad N = 25{,}000.$$

Accuracy is the official competition metric, so it is used as the model-selection criterion throughout. Because the classes are almost perfectly balanced (Section 3), accuracy is also a sensible metric here and a trivial majority-class predictor would obtain only ≈ 50%.

Constraints and rules that shaped the design:

- Only the labelled training data may be used — test labels, the public IMDB release and any external sentiment lexicon are off limits, so the models must learn sentiment from the 25,000 training reviews. General-purpose pre-trained models are allowed (the Kaggle rules restrict only their licence, see the next point). As a design decision of this project, only base checkpoints are used: checkpoints already fine-tuned on IMDB or other sentiment data, and instruction- or chat-tuned language models, may have been trained on the labels of our test reviews, so the code refuses such model names. The one exception is Qwen3.8-27B in Stage 7, which is released only in post-trained form; Section 4.13 explains why it was allowed and how this limits the interpretation of its results.
- Every library and pre-trained model must carry an open-source licence approved by the Open Source Initiative that does not restrict commercial use (Kaggle competition rule 6.c). All used here do: NumPy and pandas (BSD-3-Clause), PyTorch (BSD-3-Clause), transformers, peft, accelerate and sentencepiece (Apache-2.0), DeBERTa-v3-large and RoBERTa-large (MIT), ELECTRA-large, Qwen3-8B-Base and Qwen3-14B-Base (Apache-2.0).
- The submitted code must regenerate the leaderboard predictions exactly; the pipeline therefore fixes every random seed, logs file hashes and writes the submission directly.

- The Kaggle file must contain the header id,sentiment, every one of the 25,000 test ids exactly once, and only the two label strings.

3. Dataset Description and Analysis

3.1 Files and basic statistics

File	Records	Columns	Notes
train.csv	25,000	id, review, sentiment	ids 1–25,000; 12,528 positive / 12,472 negative
test.csv	25,000	id, review	ids 25,001–50,000; no labels
sample_submission.csv	25,000	id, sentiment	placeholder predictions (all positive); defines the required format

Key observations from the exploratory analysis (results/eda.json), all computed by the submitted code:

- Class balance. 50.1% of training reviews are positive (Figure 1), so no re-weighting or resampling is needed and accuracy is unbiased between classes.
- Completeness and duplicates. There are no missing values in either file. 104 training rows repeat an earlier training text; together they form 102 groups of identical reviews, and every group carries a single label. 222 distinct texts occur in both files — 227 test rows, because a few of them are repeated within the test set (230 after removing HTML tags, whitespace and case differences). Duplicates were kept for training: they are genuine repeated submissions in IMDB and removing them changes nothing material. Stage 4 uses the test copies directly (Section 4.10).
- Length. Reviews are long and heavy-tailed — mean 232 words, median 173, range 6–2,278 (Figure 2). The test distribution is virtually identical (mean 231 words), confirming the random split. Positive and negative reviews have almost the same length (means 233.31 vs 229.91 words), so length itself carries no label information.
- HTML noise. 58% of reviews contain literal
 line-break tags and many contain HTML entities (& etc.) — these must be stripped so that tokens such as "movie.<br" do not pollute the vocabulary.
- Vocabulary size. After cleaning, with terms appearing in at least two documents, there are 48,012 unigrams, 434,924 uni+bigrams and 921,043 uni+bi+trigrams. The document–term matrix is therefore extremely sparse (≈ 0.1% non-zero), which motivates the custom CSR representation in Section 5.

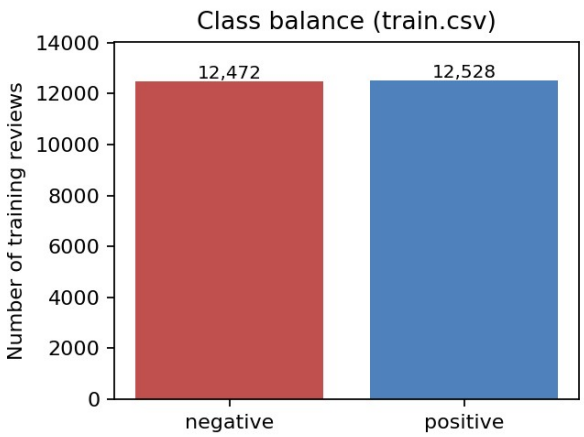


Figure 1. Class distribution of the 25,000 training reviews.

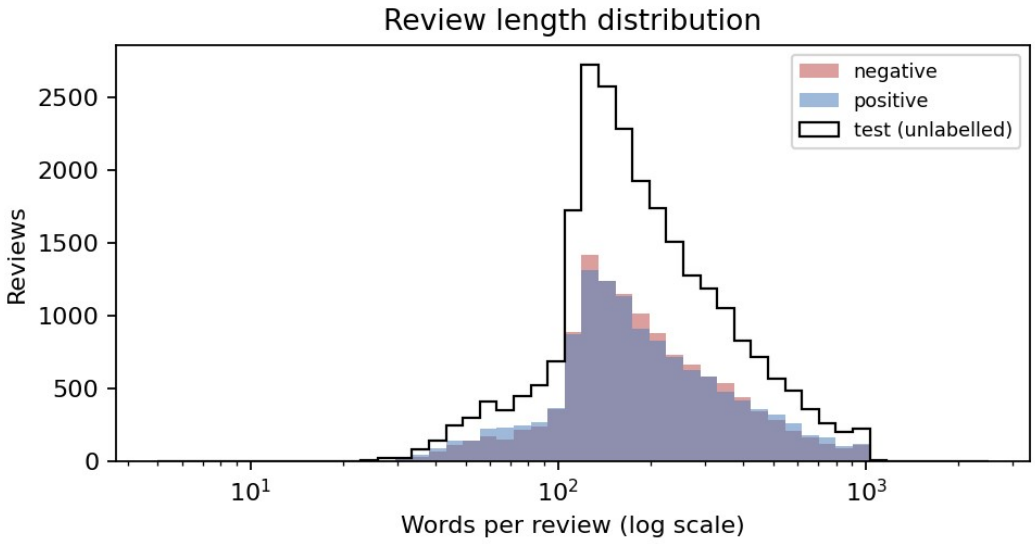


Figure 2. Words per review (log scale) for the two training classes and for the unlabelled test set.

3.2 Which phrases carry sentiment?

To see what the data can offer a bag-of-n-grams model, I ranked all unigrams and bigrams occurring in at least 20 reviews by their naive-Bayes log-count ratio, log(P(term | positive) / P(term | negative)) (Figure 3). The most negative phrases are "awful the", "terrible movie", "avoid at", "movie sucks", "this turkey", "1 out"; the most positive are "are amazing", "blew me", "simply amazing", "9 10", "rating 8", "amazing job". Two patterns stand out and directly informed the modelling choices:

- Explicit numeric ratings embedded in the text ("9 10", "1 out", "rating 8", "grade d") are among the strongest signals — reviewers often write their score — so digits must be kept as tokens rather than discarded.
- Many of the strongest cues are multi-word ("avoid at", "complete waste", "blew me", "potential but"), and negation ("not funny", "not worth the") flips meaning. Unigrams alone cannot represent these, which is why bigrams and trigrams were tested.

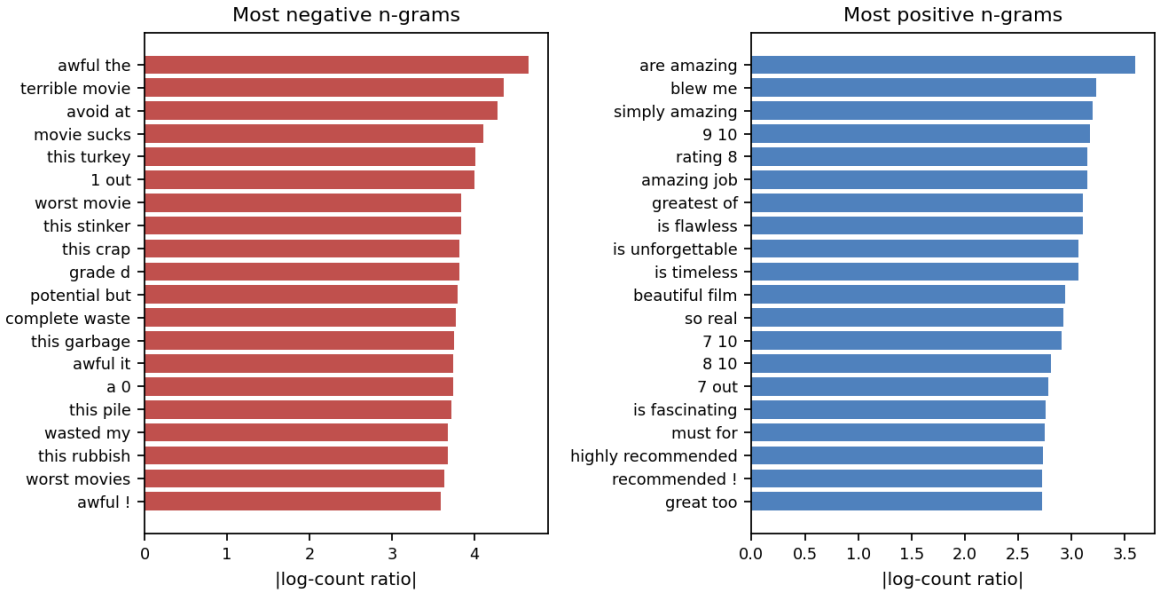


Figure 3. The 20 most negative and most positive uni/bigrams in the training data by |log-count ratio| (min. document frequency 20).

4. Methodology

4.1 Overall approach

The approach follows the classical, strong baseline for polarity classification described by Wang & Manning (2012): represent each review as a sparse bag of word n-grams and fit a regularised linear classifier. Three model families were compared under identical features and the same cross-validation folds so that the comparison isolates the effect of the classifier:

1. **Multinomial naive Bayes (MNB).** A generative model with Laplace smoothing $\alpha = 1$, tested on raw counts and on binarised presence features. For movie-review polarity, presence features work better than counts (Pang, Lee & Vaithyanathan, 2002), and binary features also help naive Bayes in other text classification tasks such as spam filtering (Metsis et al., 2006).
2. **TF-IDF + logistic regression (LR).** Sub-linear term frequency $\log(1 + tf)$, smoothed inverse document frequency and L2 row normalisation, followed by L2-regularised logistic regression with inverse regularisation strength $C \in \{4, 16, 64\}$.
3. **NB-LR.** Binarised n-gram features are first scaled by the naive-Bayes log-count ratio r (so each feature's sign and magnitude already encode its class evidence), then logistic regression is trained on the scaled features, and finally the learned weights w are interpolated towards their mean: $w \leftarrow (1 - \beta) \cdot \text{mean}(|w|) + \beta \cdot w$. $\beta = 1$ is plain LR on NB-scaled features; smaller β pulls the model towards the NB solution and acts as an extra regulariser. Grid: $C \in \{0.1, 0.3, 1\}$, $\beta \in \{0.25, 0.5, 0.75\}$.

Each family was run with maximum n-gram order 1, 2 and 3, giving 42 configurations in total. Deep models were deliberately left out of this first stage: the aim was a transparent, CPU-only reference in which every component is implemented and understood. The literature (Wang & Manning, 2012; Mesnil et al., 2015) reports that NB-weighted n-gram models come within a few points of neural models on this dataset; Stage 2 (Section 4.4) measures that gap directly.

4.2 Text pre-processing

Pre-processing was kept deliberately light, because every step that removes information was checked against the data analysis in Section 3:

- Unescape HTML entities, remove
 and any other tags, remove URLs, collapse whitespace, lower-case.
- Tokenise with the regular expression [a-z0-9]+(?:'[a-z]+)?|[!?] — this keeps digits (ratings), keeps contractions such as don't and isn't as single tokens (so negation is preserved), and keeps "!" and "?" as tokens because they are themselves sentiment cues.
- No stop-word removal and no stemming. Stop words participate in informative n-grams ("not worth the", "one of the best"), and stemming conflates e.g. "terrible" with "terribly" with little gain for a model that has 25,000 documents to learn from.
- Vocabulary pruning: keep n-grams that occur in at least 2 training documents (min_df = 2) and at most 95% of them (max_df = 0.95). Crucially, the vocabulary is rebuilt from the training part only inside every cross-validation fold, so validation reviews never influence feature selection.

4.3 Evaluation protocol

All 42 configurations were evaluated with stratified 5-fold cross-validation (seed 42) on the 25,000 training reviews, reporting the mean and standard deviation of the per-fold accuracy. The out-of-fold predictions of the chosen model (one prediction per training review, each made by a model that never saw that

review) are used for the confusion matrix, the per-segment analysis and the error inspection in Section 6. The best configuration was then refitted on all 25,000 reviews to predict the test set. No test labels or leaderboard feedback were used for any modelling decision in Stage 1, so the CV estimate is an honest forecast of leaderboard accuracy. How the public leaderboard was used in the later stages is stated in Section 6.11.

4.4 Stage 2: fine-tuning a pre-trained transformer

The error analysis in Section 7.1 shows that NB-LR's remaining errors come from sarcasm, contrast ("I expected to hate it, but …") and reviews whose verdict is one short sentence after a long digression. All of these require reading words in context. Transformers (Vaswani et al., 2017) are built for exactly this: self-attention lets the representation of every token depend on all other tokens in the review. Pre-trained on large general-domain corpora and then fine-tuned on a labelled task (Devlin et al., 2019), they are the strongest known approach to IMDB polarity (Sun et al., 2019).

I chose DeBERTa-v3-large (He et al., 2023), whose architecture is explained in Section 4.5. It gives consistently strong results on English classification benchmarks. Only the general pre-trained checkpoint was used; the code refuses model names that indicate a sentiment-fine-tuned checkpoint (Section 2). Fine-tuning setup:

- **Input.** HTML tags are removed and entities unescaped, but case and punctuation are kept, because the model's sub-word tokenizer was pre-trained on natural text. The mean review length is 257 tokens and 12% of reviews exceed the 512-token limit. These are truncated with the head+tail strategy of Sun et al. (2019): the first 128 and the last 382 tokens are kept, because opinions concentrate at the start and the end of a review.
- **Validation.** A stratified 10% hold-out (2,500 reviews, seed 42) is never trained on. The model is evaluated on it every half epoch and the best checkpoint is kept; the same reviews are later used to compare ensembles (Section 4.6). Five-fold CV as in Stage 1 would cost five times the GPU time, for a model whose hold-out accuracy already has a 95% confidence interval of about ±0.7 points. (Stage 3 later paid this price, Section 4.7.)
- **Optimisation.** A two-class classification head on the first token, cross-entropy loss, and AdamW (Loshchilov & Hutter, 2019) with peak learning rate 1×10^{-5}, weight decay 0.01, linear warm-up over the first 10% of updates and linear decay to zero. The effective batch is 16 reviews (8×2 gradient-accumulation steps), training runs for 2 epochs with gradient clipping at 1.0, in bfloat16 mixed precision. These are standard values for large encoders: the small learning rate and the warm-up protect the pre-trained weights from being overwritten early in training.
- **Further runs for an ensemble.** After the first run (seed 42) had been evaluated, the same recipe was run twice more with seeds 1 and 2, and once with a different pre-trained model, RoBERTa-large (Liu et al., 2019), using the same hyper-parameters. A new seed changes the random classification head and the order of the training batches, so it shows how much the result depends on chance. RoBERTa was chosen as a second architecture (Section 4.5), in the hope that it makes more different errors. All runs use the same hold-out split (seed 42), so their hold-out scores can be compared and combined review by review.

4.5 How DeBERTa-v3 classifies a review

NB-LR gives every n-gram one fixed weight, whatever its context. A transformer instead computes, for every token, a vector that depends on the whole review, so "good" after "not" — or a verdict after "but"

— can be represented differently from the same word elsewhere. Figure 4a shows the five steps from text to prediction; the numbers are those of DeBERTa-v3-large. (Figure 4b in Section 4.9 shows the same steps for the Stage-4 language model.)

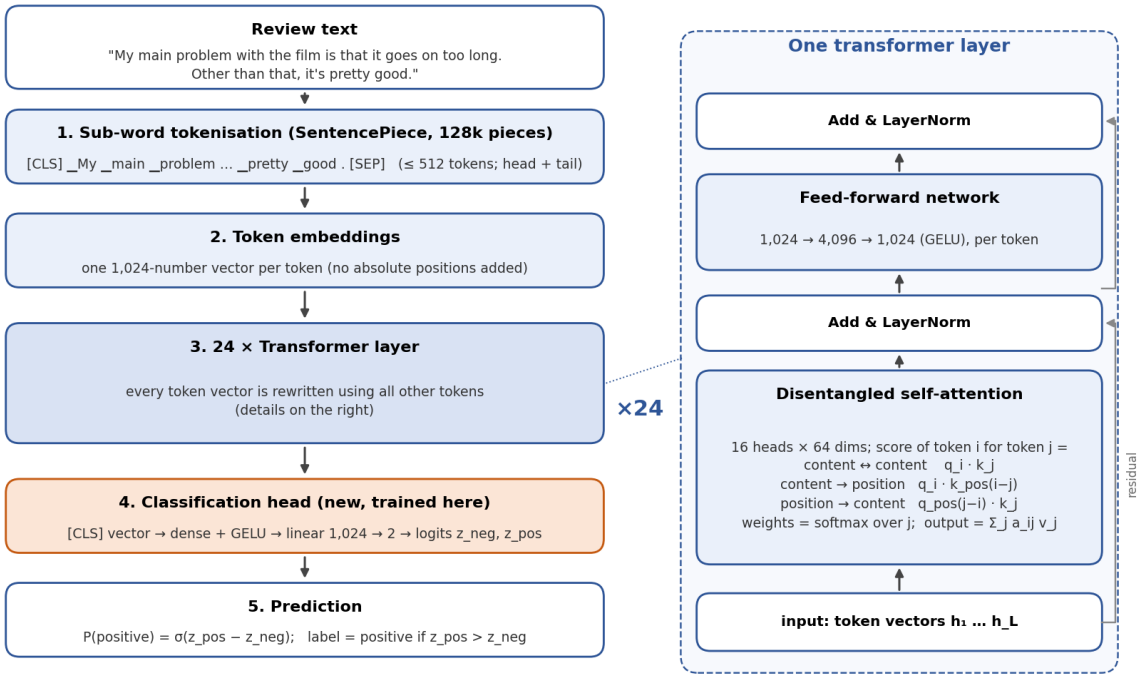


Figure 4a. DeBERTa-v3-large as used here: data flow from review to prediction (left) and the structure of one of its 24 layers (right).

1. Sub-word tokenisation. A SentencePiece model (Kudo & Richardson, 2018) with a vocabulary of 128,000 pieces splits the text. Frequent words are single pieces; rare words, names and misspellings are split into several known pieces, so no word is ever out of vocabulary. A [CLS] token is added at the start and a [SEP] token at the end. Because the cost of attention grows with the square of the length, the pre-trained model accepts at most 512 tokens, which is why long reviews are truncated (Section 4.4).

2. Embeddings. Each piece is replaced by a learned vector of d = 1,024 numbers. Unlike BERT, DeBERTa adds no absolute position information to these vectors: word order is handled inside attention, through relative positions (step 3).

3. Transformer layers. Twenty-four identical layers (Vaswani et al., 2017) each rewrite all token vectors. A layer has two parts, each followed by a residual connection (the input is added back) and layer normalisation:

- **Self-attention.** From each token vector h_i the layer computes a query q_i, a key k_i and a value v_i by three learned linear maps. Token i attends to token j with weight $a_{ij} = \text{softmax}_j(s_{ij})$, and its new vector is the weighted sum $\Sigma_j\, a_{ij}\, v_j$. Thus every token can collect information from any other token, however far away. This is done in 16 parallel "heads" of 64 dimensions each, which can specialise, for example in negation scope or in linking a verdict to the film it refers to.
- **Disentangled attention (He et al., 2021).** In DeBERTa the score combines what the tokens are and where they are relative to each other: $s_{ij} = [q_i \cdot k_j + q_i \cdot k^{pos}(i-j) + q^{pos}(j-i) \cdot k_j] / \sqrt{(3 \cdot 64)}$. The first term compares contents ("is token j a negation word?"), the other two use learned vectors for the relative distance between i and j ("the token two places before me"). Separating content from

position helps with patterns such as "not … good", where meaning depends on both the words and their distance.

- **Feed-forward network.** Each token vector then passes on its own through a two-layer network (1,024 → 4,096 → 1,024 with GELU activation), which transforms the information gathered by attention.

4. Pre-training (done once by the model authors, not here). The 24 layers were pre-trained on 160 GB of general English text (Wikipedia, books and web pages) by replaced-token detection (Clark et al., 2020). A small generator network replaces about 15% of the tokens with plausible alternatives, and the main network must decide, for every token, whether it is original or replaced. To do this well it must model grammar, word meaning and context, without any sentiment labels. DeBERTa-v3 adds gradient-disentangled embedding sharing, which stops the generator's and the discriminator's training signals from pulling the shared word embeddings in opposite directions.

5. Classification head and fine-tuning (done here). A new, randomly initialised head takes the final vector of the [CLS] token, passes it through a dense 1,024 → 1,024 layer with GELU activation and a linear layer 1,024 → 2, and outputs two logits z_neg and z_pos. The predicted probability is P(positive) = softmax(z)_pos = σ(z_pos − z_neg), and the margin m = z_pos − z_neg (the log-odds) is the score used for the ensemble. Fine-tuning minimises the cross-entropy −log P(true label) on the 22,500 training reviews. Back-propagation updates the new head and also all pre-trained weights, gently, with the AdamW settings of Section 4.4. Because the network already represents syntax, negation and word meaning, fine-tuning only has to learn which of these internal features signal a positive or negative opinion. This is why the model reaches 95.6% hold-out accuracy after only half an epoch (Section 6.3).

Model size. Each layer holds about 12.6 million weights ($4 \times 1{,}024^2$ for attention, $2 \times 1{,}024 \times 4{,}096$ for the feed-forward network), so the 24 layers hold about 302 million. The 128,000 × 1,024 embedding table holds another 131 million, giving about 435 million parameters in total. Stored as 32-bit numbers, that is 1.7 GB: this is the size of the saved checkpoint, and the reason it is not included in the Moodle ZIP (Section 5.4). By comparison, NB-LR has 921,043 weights.

RoBERTa-large, the second architecture. RoBERTa (Liu et al., 2019) has the same overall structure — 24 layers, d = 1,024, 16 heads, about 355 million parameters (a 1.4 GB checkpoint) — but differs in the three places where DeBERTa-v3 innovates: it adds absolute position embeddings to the token vectors instead of using disentangled relative attention, it splits text with a byte-level BPE vocabulary of 50,265 pieces, and it was pre-trained by masked-language modelling (predict 15% hidden tokens) on 160 GB of text rather than by replaced-token detection. Different pre-training and tokenisation can lead to different errors, which is what an ensemble needs. With absolute positions and a smaller vocabulary it is also faster: training took 6 minutes on our GPU instead of 11–21.

4.6 Combining the models

NB-LR and the transformers work very differently — one counts phrases, the others read context — so their errors may be partly complementary; the hold-out scores of NB-LR and DeBERTa correlate at only r = 0.79. code/ensemble.py takes the hold-out scores of any number of runs plus NB-LR's out-of-fold scores, so that no model has seen the 2,500 reviews used for the comparison, and compares three options: (a) the best single model, (b) the mean of the standardised scores of all models (each score minus its hold-out mean, divided by its hold-out standard deviation, so that no model dominates by scale), and (c) a stacked logistic regression with one weight per model. The options are compared by 5 × repeated stratified 10-fold CV inside the hold-out. Stacking is chosen only if it wins by more than 0.05 points,

because it fits extra parameters on just 2,500 reviews. Which models enter the ensemble was fixed in advance (all of them), not chosen by their hold-out score, so the hold-out is not over-fitted by trying many subsets (Section 6.4).

4.7 Stage 3: cross-validation instead of one hold-out

Stage 2 left two problems. First, the 2,500-review hold-out is too small to compare good models: its standard error is about 0.35 points, so a real gain of 0.3 points is invisible in it (Section 6.4 found a 0.5-point spread between seeds of the same model). Second, every Stage 2 transformer was trained on only 22,500 of the 25,000 labelled reviews. Stage 3 solves both with the protocol that Stage 1 already used:

- **Partition.** One stratified 5-fold partition of the 25,000 training reviews (seed 42) is shared by every model family. It is built from the same per-class shuffles as the Stage 2 hold-out, so fold 0's 5,000 validation reviews contain the 2,500 Stage 2 hold-out reviews, which allows a like-for-like check against Stage 2.
- **Out-of-fold (OOF) predictions.** For each fold j a model is fine-tuned on the other four folds (20,000 reviews) and predicts the 5,000 reviews of fold j and all 25,000 test reviews. Every training review thus receives exactly one prediction from a model that never saw it. Pooled over 25,000 reviews, the OOF accuracy has a standard error of about 0.11 points, three times finer than the hold-out. Because all families share the partition, two families can be compared review by review with an exact McNemar test (McNemar, 1947), which uses only the reviews on which they disagree.
- **Test predictions.** The test margin of a family is the mean of its five fold models' margins. The five models together have seen all 25,000 labels, and averaging them reduces the variance of a single run, so the averaged file is expected to be slightly better than the OOF accuracy, which describes a single fold model.

Three experiments were run with this protocol (Section 6.5): (a) DeBERTa-v3-large with the Stage 2 recipe; (b) ELECTRA-large (Clark et al., 2020), a third encoder, with the same hyper-parameters, which were not re-tuned for it; and (c) a re-run of the DeBERTa recipe started explicitly from the general pre-trained checkpoint, after a screening of four learning-rate/epoch/truncation settings on the 2,500-review hold-out. An earlier attempt at self-training is also reported: the four Stage 2 transformers labelled the test reviews, and the confident labels were added to the training data (Section 6.5).

Combining families (code/ensemble_cv.py). With 25,000 OOF rows the combination can be chosen far more reliably than on 2,500. The rule below was fixed before any Stage 4 result was seen:

- **s-mean:** divide each family's margins by their OOF standard deviation and average them. Nothing is subtracted, so every family keeps its own decision threshold at zero, and averaging a family with itself reproduces it exactly. The Stage 2 combiner (z-mean) also subtracted each family's mean margin, which shifts the thresholds: averaging NB-LR with itself under z-mean changed 575 of 25,000 predictions, against none under s-mean. It is therefore reported for comparison only, as are the raw mean and the rank mean (which forces exactly 50% positive predictions).
- **stack:** an L2-regularised logistic regression on the margins (stacked generalisation; Wolpert, 1992), the only combiner that uses labels. Its accuracy is therefore estimated by 10 × repeated stratified 10-fold CV inside the 25,000 OOF rows.
- **Rule:** use s-mean if it beats the best single family on the OOF rows, otherwise that family; use the stack only if it beats this choice by more than one standard error (about 0.11 points).

4.8 Stage 4: why a large language model

Stage 3 showed that the encoders had saturated. On the 25,000 OOF reviews the margins of DeBERTa correlate with those of ELECTRA at r = 0.984 and with those of its own re-run at 0.994; RoBERTa in Stage 2 was at 0.98. All three are encoders of 335–435 million parameters, pre-trained on similar English text (Wikipedia, books, news and web pages) with similar objectives, so they learn the same things and make the same mistakes. A further encoder, seed or hyper-parameter setting therefore adds little information. A useful new model has to know something the encoders do not.

A decoder-only large language model (LLM) is the most different model available. Qwen3-8B-Base (Yang et al., 2025) was pre-trained by next-token prediction on about 36 trillion tokens in 119 languages, two to three orders of magnitude more text than DeBERTa-v3. That text includes far more film criticism, background knowledge about films and actors, and examples of irony. The hypothesis was that such a model would recognise sarcasm, mock praise and comparisons with other films better than the encoders, and would make partly different errors.

Three constraints shaped the choice of model:

- **Base checkpoint only.** The post-trained (chat) Qwen3 models went through instruction tuning and reinforcement learning on data that may contain sentiment tasks, and half of our test reviews come from the original IMDB training split. The code accepts only pre-trained checkpoints and refuses Qwen3 names that do not end in -Base.
- **Licence.** Qwen3 is released under Apache-2.0 (OSI-approved, commercial use allowed; Section 2). Models whose licences are not OSI-approved, such as the Llama family, were not considered for this reason.
- **Memory.** 8 billion parameters is the largest Qwen3 base model whose LoRA fine-tuning fits into the 24 GB of the GPU used here (peak 20.2 GB, Section 5.5).

A known caveat is that a web-scale pre-training corpus may contain IMDB review pages, possibly with their star ratings. This cannot be ruled out. Its effect is bounded by the zero-shot check in Section 4.9: before any fine-tuning the model is 94.7% accurate (on 1,280 validation reviews, 256 per fold), well below the 97.0% it reaches after fine-tuning on the training folds. The test labels are never used.

4.9 How the Qwen3 classifier works

The steps from text to prediction parallel those of DeBERTa in Section 4.5 and are shown in Figure 4b. The two models differ in three essential ways: Qwen3 reads the review left to right only, it answers a question appended to the review, and all of its 8 billion weights stay frozen.

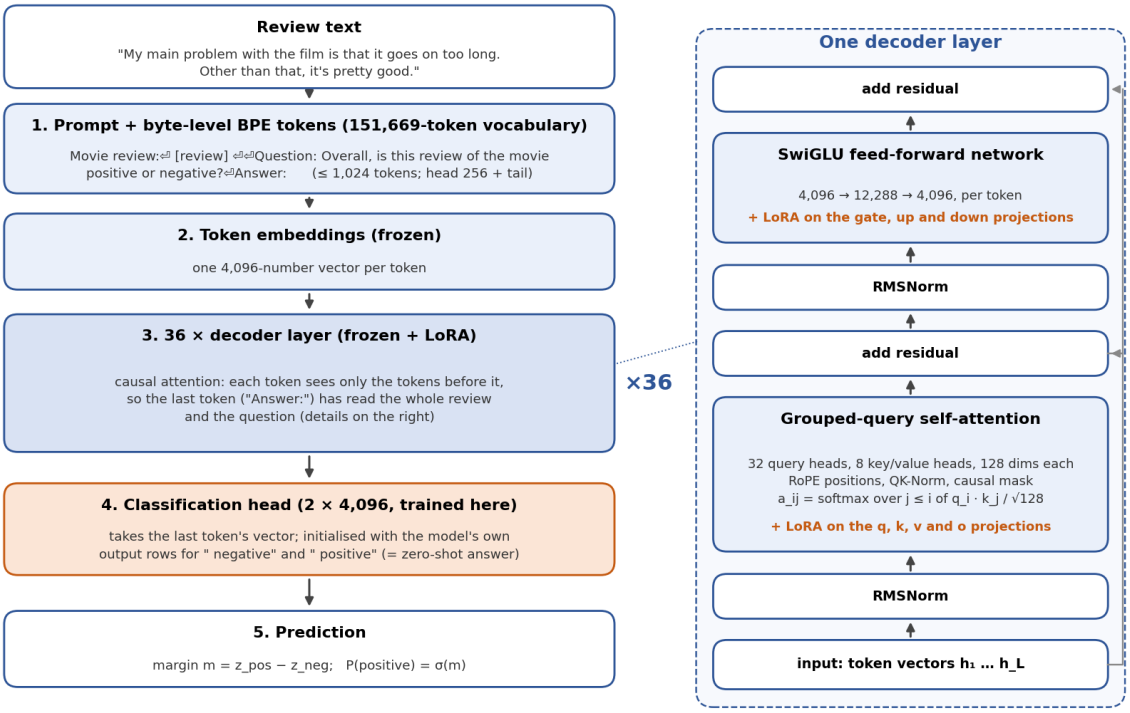


Figure 4b. Qwen3-8B-Base with LoRA as used here: data flow from review to prediction (left) and the structure of one of its 36 decoder layers (right). Orange: the parts that are trained.

1. Prompt and tokenisation. The review (HTML removed) is wrapped in a fixed prompt: "Movie review:" + review + "Question: Overall, is this review of the movie positive or negative? Answer:". It is split by Qwen's byte-level BPE tokenizer (151,669 tokens), which can encode any byte sequence, so nothing is ever out of vocabulary. The input is limited to 1,024 tokens including the 20 prompt tokens. Longer reviews keep their first 256 and their last tokens, as in Stage 2. The mean length is 304 tokens, and only 1.8% of the training reviews and 1.7% of the test reviews are truncated, compared with 12% at DeBERTa's 512-token limit.

2. Decoder layers. Each token becomes a vector of 4,096 numbers, which passes through 36 decoder layers. Each layer applies RMSNorm (Zhang & Sennrich, 2019), grouped-query self-attention, a residual connection, RMSNorm again, a SwiGLU feed-forward network (4,096 → 12,288 → 4,096; Shazeer, 2020) and a second residual connection. Grouped-query attention (Ainslie et al., 2023) has 32 query heads that share 8 key/value heads of 128 dimensions each, which cuts the memory needed for keys and values. Positions enter through rotary embeddings (RoPE; Su et al., 2024), which rotate queries and keys by an angle proportional to their position, so attention scores depend on relative distance. QK-Norm normalises queries and keys, which stabilises training.

3. Causal attention, and why the last token is used. Unlike DeBERTa, every token may attend only to itself and to earlier tokens (a causal mask). The [CLS] vector of an encoder sees the whole input, but here only the vector of the final prompt token — the colon after "Answer" — has read the whole review and the question. This is the position at which a language model would write its answer, so it is the natural summary of the review and is used as the review representation.

4. Classification head, initialised as the zero-shot answer. A linear layer maps the 4,096-number vector to two logits. Its two rows are copied from the model's own output layer for the tokens " negative" and " positive", and the bias starts at zero. Before any training, therefore, the margin m = z_pos − z_neg is

exactly the model's zero-shot preference between answering "positive" and "negative". This gives a meaningful check at update 0: on 256 validation reviews per fold, the untrained classifier was already 93.8 –95.7% accurate (AUC 0.980–0.995). Fine-tuning has to remove the remaining errors, rather than learn a randomly initialised head from scratch as in DeBERTa.

5. LoRA fine-tuning. Full fine-tuning of 8 billion weights with AdamW would need about 16 bytes per weight for the weights, gradients and optimiser moments, over 120 GB in total. LoRA (Hu et al., 2022) instead freezes every pre-trained matrix W and learns a low-rank update: an adapted projection computes $W x + (\alpha / r) \cdot B A x$, where A is $r \times d_in$, B is $d_out \times r$, and B starts at zero, so training begins exactly at the pre-trained model. With rank $r = 16$ and $\alpha = 32$, adapters on all seven projections of every layer (query, key, value, output, gate, up, down) add 43.6 million trainable weights, 0.57% of the 7.61 billion. The frozen weights are stored once in bfloat16 (14.3 GB on the GPU), and only the adapters and the 8,194-weight head need gradients and optimiser state. A saved adapter takes about 175 MB instead of 16 GB.

6. Training. The loss is the cross-entropy of the two logits. The optimiser is AdamW with peak learning rate 1×10^{-4} for the adapters and 2×10^{-4} for the head, weight decay 0.01, linear warm-up over the first 5% of updates and cosine decay to zero. One epoch is 1,250 updates of 16 reviews, with gradient clipping at 1.0, bfloat16 autocast and gradient checkpointing. Reviews of similar length are grouped into micro-batches of at most 4,096 tokens, so little computation is spent on padding. These are common LoRA settings and were fixed in advance; there was no hyper-parameter search, and one seed (42) was used. The final weights are used, and there is no checkpoint selection on the validation fold, so the Qwen3 OOF estimate is not optimistically biased. (The DeBERTa runs kept the best of four evaluations, which favours DeBERTa slightly in the comparison.) One evaluation half-way through the epoch is logged for monitoring only.

7. Saved model = submitted model. After training the adapter and head are saved, the model is freed, and the base model plus adapter are re-loaded from disk. All validation and test predictions come from the re-loaded copy. A reload check compares the two copies on 256 reviews; the largest margin difference was 0.0000 in all five folds.

4.10 Final combination and exact-match backfill

The Qwen3 and DeBERTa families are combined with the rule of Section 4.7, applied to their 25,000 OOF rows. A last step, which uses no test labels, then exploits a property of the data found in Section 3.1:

- 227 test reviews are character-for-character identical to a training review. An IMDB label is derived from the star rating that the author gave together with the text (Maas et al., 2011), and the 102 groups of identical texts inside train.csv all carry a single label, so the training label is the best available prediction for its exact copy, even when the text seems to say otherwise.
- code/apply_exact_match.py therefore replaces the model's prediction for these test reviews by the label of their training twin. It uses only training labels, refuses to run if identical training texts had conflicting labels, and writes an audit file listing every changed id.

4.11 Stage 5: a larger language model on a rented GPU

Stage 4 left one question open: is the gain over DeBERTa a property of large language models that grows with size, or was 8 billion parameters already enough? The next base model in the same family, Qwen3-14B-Base (Apache-2.0, 13.90 billion parameters, 40 decoder layers of width 5,120), answers this most cleanly, because it was pre-trained on the same data with the same tokenizer and prompt format. Its

bfloat16 weights alone take 28 GB, more than the 24 GB GPU of Stages 2–4, so Stage 5 ran on a rented cloud machine with one NVIDIA RTX PRO 6000 (96 GB).

To make the comparison with the 8B model paired and fair, everything except the model was kept the same: the fold assignment (seed 42), the prompt, truncation, LoRA rank 16 on all seven projections, learning rates, batch of 16 reviews, one epoch, and the use of the final weights. LoRA trains 64.2 million weights (0.46% of the model). The only other change is that gradient checkpointing was switched off, because the 96 GB card has room for the activations; this changes speed and memory but not the computed gradients.

The selection rule was written down before the run (Section 5.6). The Stage 5 combination would replace the Stage 4 rule choice only if its estimated OOF accuracy exceeded the 97.02% of Qwen3-8B alone. The combination itself was chosen by the rule of Section 4.7, applied to the three families DeBERTa, Qwen3-8B and Qwen3-14B. The exact-match backfill of Section 4.10 is applied afterwards, as in Stage 4. The public score was used only as a check that the file is not broken (it had to be at least 0.970).

4.12 Stage 6: a differently pre-trained model on a rented 80 GB GPU

Stage 5 left a different question open: is the gain over DeBERTa specific to Qwen3, or does another base language model reach it too, with errors different enough to help a combination? The two Qwen3 models agree on 98.82% of the OOF reviews (Section 6.8), so a third model of the same family would add little. Stage 6 therefore used Mistral-Small-24B-Base-2501 (Mistral AI, 2025), an Apache-2.0 base model released in January 2025. It was pre-trained by a different company on different data, with a different tokenizer (Tekken, 131,072 tokens). The training log counts 22.99 billion weights and a hidden width of 5,120. In bfloat16 it takes 43.0 GB on the GPU, so it ran on a rented AutoDL machine with one NVIDIA H800 PCIe (80 GB), with gradient checkpointing switched on.

Everything except the model was kept as in Stages 4 and 5: the fold assignment (split seed 42), the prompt, head + tail truncation at 1,024 tokens, LoRA rank 16 on all seven projections, learning rates, batches of 16 reviews, one epoch, and the final weights. LoRA trains 92.4 million weights (0.40%). Two details differ. First, the tokenizer is loaded with fix_mistral_regex=True, the option that transformers asks for with this model; before training, a check script confirmed that the tokenizer then splits 2,004 of 2,004 test texts exactly like Mistral's reference tokenizer (mistral_common). Second, the training seed of fold 3 is 43 instead of 42, because the first run of that fold diverged (Section 6.9).

The rule was again written down before the run. The combination is chosen by the rule of Section 4.7 over the four families DeBERTa, Qwen3-8B, Qwen3-14B and Mistral-24B, and the backfill of Section 4.10 is applied, giving file D. D replaces C only if the chosen combination's OOF estimate is at least 97.254%: C's 97.204% plus 0.05 points. The margin is needed because adding a family to s-mean changes the weights even when it brings no new information. In a simulation in which a copy of the 14B predictions stood in for the new model, the four-family s-mean estimated 97.212%, already above C. The public score was again only a check that the file is not broken (at least 0.970).

4.13 Stage 7: a newer architecture, available only post-trained

After Stage 6, three language models from two developers shared most of their errors. Stage 7 tested one last kind of difference: architecture. Qwen3.8-27B (Qwen Team, 2026; Apache-2.0) is a newer Qwen model whose text part has 64 layers, of which 48 are Gated DeltaNet layers (a form of linear attention; Yang, Kautz & Hatamizadeh, 2025) and only 16 are ordinary attention layers. The training log counts 25.74 billion weights and a hidden width of 5,120. The checkpoint is a vision-language model; the code loads it

whole and uses only its text part. In bfloat16 it takes 48.2 GB on the GPU, so it ran on a rented machine with one NVIDIA RTX PRO 6000 (96 GB).

An exception to the base-model rule. Qwen3.8-27B is released only in post-trained form; there is no -Base checkpoint. Stages 4–6 accepted only base checkpoints (Section 4.8). This is a project rule, not a Kaggle rule, and the licence is allowed (Section 2). For this one preset the code allows the post-trained checkpoint by name; every other instruction or chat model is still refused. The post-training data are not published and may include sentiment tasks or IMDB-style reviews, so its results carry a contamination risk that the base models do not. The zero-shot check bounds the risk somewhat. Before fine-tuning the model is 88.9% accurate on the 1,280 checked reviews (AUC 0.979), lower than the base models (95.2% for Qwen3-14B, 95.6% for Mistral-24B). A model trained on these labels would be expected to do better than that. It cannot rule out contamination.

Everything else was kept as in Stages 4–6: the fold assignment (split seed 42), the prompt, head + tail truncation at 1,024 tokens, LoRA rank 16, learning rates, batches of 16 reviews, one epoch, the final weights, and training seed 42 in every fold. LoRA is placed on every linear layer of the text part (496 layers, including the projections of the DeltaNet layers), not on the vision part or the output head. It trains 116.7 million weights (0.45%). Gradient checkpointing was on.

The rule was written down before the run. The combination is chosen by the rule of Section 4.7 over five families (DeBERTa, Qwen3-8B, Qwen3-14B, Mistral-24B, Qwen3.8-27B), and the backfill of Section 4.10 is applied, giving file E. E replaces D only if the chosen combination's OOF estimate is at least 97.384%: D's 97.304% plus 0.08 points. The margin is larger than in Stage 6 because a simulation in which a copy of the Mistral-24B predictions stood in for the new model already gave a five-family s-mean of 97.332%. The public score was again only a check that the file is not broken (at least 0.970).

5. Implementation Details

Stage 1 is implemented in Python 3.10.12 using only numpy 2.2.6, pandas 2.3.3 and matplotlib (figures). The code is organised as a small package plus one driver script; the Stage 2 files are described in Section 5.4, the Stage 3–4 files in Section 5.5 and the rented-GPU runs of Stages 5–7 in Sections 5.6–5.8.

File	Purpose
code/sentiment/sparse.py	CSRMatrix: a compressed-sparse-row matrix supporting X·w, X^T·r, row/column scaling, L2 row normalisation and binarisation — all vectorised with numpy.bincount / numpy.repeat so no Python loops run over the ~8 million non-zeros.
code/sentiment/features.py	clean_text, tokenize, n-gram generation; NgramCountVectorizer (single tokenisation pass in fit_transform, then document-frequency pruning; transform = Counter → CSR) and TfidfTransformer.
code/sentiment/models.py	MultinomialNB, LogisticRegression and NBLogisticRegression, plus the L-BFGS optimiser (two-loop recursion, memory 10, Armijo backtracking line search).
code/sentiment/evaluation.py	accuracy, confusion matrix, precision/recall/F1, stratified k-fold splitter.
code/run_pipeline.py	Driver with three stages — eda, cv, final — producing every table, figure and the Kaggle submission.csv. CV work is check-pointed per (fold, n-gram order) so an interrupted run resumes.
code/sentiment/plots.py	All matplotlib figures; imported only when plots are requested (--no-plots skips it).
tests/test_sentiment.py	24 unit tests: 17 NumPy-only numerical checks of the sparse algebra, vectoriser, models and metrics, plus 7 optional comparisons against scikit-learn/SciPy.

5.1 Logistic regression and optimisation

The LR objective is the mean logistic loss plus an L2 penalty, $J(w, b) = (1/n)\ \Sigma_i \log(1 + \exp(-t_i(w \cdot x_i + b))) + \|w\|^2 / (2Cn)$, with $t_i \in \{-1, +1\}$ — exactly the parameterisation used by scikit-learn (Pedregosa et al., 2011), so C values are comparable with the literature. The gradient needs one sparse product $X \cdot w$ and one transposed product $X^T \cdot (-t \cdot \sigma(-\text{margin}))$ per evaluation, each costing O(nnz). The L-BFGS optimiser (Liu & Nocedal, 1989) converges in 30–60 iterations (the final fit took 33 iterations), i.e. a few seconds even with 921,043 features. The loss is computed with numpy.logaddexp and the sigmoid via tanh for numerical stability.

5.2 NB-LR details

For NB-LR the log-count ratio is $r = \log((p/\|p\|_1)/(q/\|q\|_1))$, where p and q are the per-feature document counts in the positive and negative class plus the smoothing constant $\alpha = 1$. Features are binarised ($x \leftarrow \mathbb{1}[x > 0]$) before scaling because repeated mentions of "terrible" in one review do not make it more negative in proportion. After LR has been fitted on $X_{bin} \odot r$, the interpolation $w \leftarrow (1 - \beta) \cdot \text{mean}(|w|) + \beta \cdot w$ is applied; the effective weight on the original presence feature is therefore $w \cdot r$, which is what Figure 9b plots.

5.3 Reproducibility and run-time

- Fixed seed (42) for the fold assignment and learning-curve subsampling; the models themselves are deterministic (convex objectives, deterministic optimiser).
- run_environment.json records library versions, the SHA-256 of train.csv/test.csv and the exact command; final_model.json records the SHA-256 of the produced submission.csv.
- Run-time on a 4-core laptop CPU without GPU: vectorising 20,000 reviews takes 4 s (unigrams) to 13 s (up to trigrams); one NB fit < 0.5 s; one LR/NB-LR fit 3–6 s. The full 42-model × 5-fold study runs in ≈ 12 minutes; the final fit on 25,000 reviews takes 19 s and prediction 6 s. Peak memory is below 2 GB.
- After a later clean-up of the code (single-pass vectoriser, plotting moved to its own module), the complete 42-model × 5-fold study, the figures and the final refit were re-run: every fold accuracy and out-of-fold score is identical to the original run and the regenerated submission.csv is byte-identical (same SHA-256) to the file uploaded to Kaggle.

Reproduce the submission with:

```
pip install -r requirements.txt
python code/run_pipeline.py --data-dir data --output results               # EDA + CV + final
python code/run_pipeline.py --data-dir data --output results --stage final   # refit only
```

5.4 Stage 2 implementation

File	Purpose
code/finetune_transformer.py	PyTorch + Hugging Face transformers (Wolf et al., 2020): cleaning, sub-word tokenisation with head+tail truncation, length-grouped batching, the training loop with hold-out evaluation and checkpointing, prediction, and Kaggle-format validation. A GPU-memory probe chooses the batch size automatically. --smoke runs a 3-minute end-to-end check; --predict-only reloads the saved checkpoint.
code/ensemble.py	NumPy only: compares single / mean / stacked combinations by CV inside the hold-out and writes the ensemble submission.
code/analyze_stage2.py	Hold-out comparison of DeBERTa (seed 42) with NB-LR (Table 3, Figures 10–11) and results/stage2/stage2_summary.json.
code/analyze_ensemble.py	NumPy only: per-run statistics, score correlations, subset ablation and McNemar test for

File	Purpose
	the multi-run ensemble (Table 4, Figure 12, results/stage2/ensemble2_summary.json).
tests/test_stage2.py	26 unit tests of truncation, hold-out splitting, batching, the submission format, the ensemble maths, the analysis metrics and the pseudo-label rules.

- Environment: Python 3.12.10, PyTorch 2.11.0+cu128 (CUDA 12.8) and transformers 4.57.6 on Windows 11, with one NVIDIA GeForce RTX 5090 D v2 (24 GB). Peak GPU memory was 15.7 GB for DeBERTa-v3-large and 6.0 GB for RoBERTa-large.
- Run-time: tokenising 50,000 reviews takes 5 s. Training (2 × 1,407 updates, including five hold-out evaluations) took 13.1 minutes for the first DeBERTa run, 21.5 and 11.2 minutes for the two other seeds and 6.2 minutes for RoBERTa-large; predicting the hold-out and test sets takes 0.6–1.7 minutes. All four runs together took about an hour. Each batch groups reviews of similar length, so on average only about half of the 512-token window has to be computed (mean length 257 tokens).
- Reproducibility: each run saves its selected checkpoint as runs/<run>/best_model. At 1.4–1.7 GB each (Section 4.5) they are too large for the Moodle ZIP; they are kept by the author and can be provided on request. Everything needed to re-create them is in the ZIP: the training code, the exact command, configuration and environment of every run (run_info.json), the full training logs and all hold-out and test scores, from which code/ensemble.py and code/analyze_ensemble.py regenerate the final submission and every Stage 2 number without a GPU. Reloading the first checkpoint with --predict-only regenerated a submission.csv byte-identical to the file uploaded to Kaggle (SHA-256 cc4f338f6ed409c9…). Training from scratch is seeded, but GPU kernels are not bit-for-bit deterministic, so a re-trained model should match within a few tenths of a point rather than exactly.

Reproduce Stage 2 (GPU) with:

```
pip install -r requirements-gpu.txt      # after installing the CUDA build of PyTorch
python code/finetune_transformer.py --data-dir data --output runs/deberta
python code/finetune_transformer.py --data-dir data --output runs/deberta-large-s1 --seed 1
python code/finetune_transformer.py --data-dir data --output runs/deberta-large-s2 --seed 2
python code/finetune_transformer.py --data-dir data --output runs/roberta-large \
    --preset roberta-large
python code/ensemble.py --data-dir data --runs runs/deberta runs/deberta-large-s1 \
    runs/deberta-large-s2 runs/roberta-large --nblr-results results --output ensemble2   # CPU
```

5.5 Stages 3 and 4 implementation

File	Purpose
code/run_cv.py	Runs all K folds of a trainer (finetune_transformer.py or finetune_llm.py), one process per fold so that GPU memory is released in between. Finished folds are skipped, so an interrupted run resumes; it refuses to mix folds of different models in one folder.
code/assemble_cv.py	NumPy only: checks that the folds partition the 25,000 training reviews exactly once, pools the OOF predictions (Wilson interval, per-fold spread, accuracy by length, McNemar test against Stage 2), averages the fold models' test margins and writes the family's submission.
code/ensemble_cv.py	NumPy only: combines several families on the 25,000 OOF rows with the pre-registered rule of Section 4.7 and applies the chosen formula to the test margins; --choice forces a combiner.
code/finetune_llm.py	PyTorch + transformers + peft (Mangrulkar et al., 2022): prompt construction and head+tail truncation, LoRA set-up, head initialised from the verbaliser rows, zero-shot check at update 0, GPU-memory probe, length-grouped token-budget batching, training with out-of-memory recovery, save → free → reload → predict, reload check. --smoke

File	Purpose
	runs a 2-minute end-to-end test; --predict-only re-creates the predictions from a saved adapter.
code/compare_runs.py	NumPy only: paired comparison of two runs on the same validation reviews (the Stage 4 pilot on fold 0).
code/apply_exact_match.py	Standard library only: the exact-match backfill of Section 4.10, with an audit file of every change.
code/analyze_stage34.py, code/stage34_plots.py	NumPy only (matplotlib for figures): the Stage 3–7 numbers, tables and figures of Sections 4.8–4.13, 5.5–5.8, 6.5–6.11 and 7.3–7.5, written to results/stage34/.
code/pseudo_label.py	The self-training experiment of Section 6.5 (not used by any final submission).
tests/test_stage3.py, test_stage4.py, test_exact_match.py, …	With the Stage 1–2 tests, 130 unit tests in seven files. The new ones cover the partition, OOF assembly, McNemar direction, the combiners (a family averaged with itself must not change), batching and truncation, model-name checks, the run_cv command lines and the backfill. All pass without a GPU; the 7 optional scikit-learn comparisons are skipped if it is not installed. The GPU code paths are not unit-tested; they were checked by the smoke tests.

- Environment: Python 3.12.10, PyTorch 2.11.0+cu128, transformers 4.57.6, peft 0.18.1 and accelerate on Windows 11 with the same RTX 5090 D v2 (24 GB). Qwen3-8B-Base is loaded in bfloat16 (14.3 GB on the GPU). A memory probe before training runs the largest micro-batch, peaking at 18.9 GB reserved, and training peaked at 20.2 GB.
- Run-time per Qwen3 fold: about 45 minutes of training (about 2,260 tokens per second), 4.7 minutes for the half-epoch evaluation, 19 minutes to predict 5,000 validation and 25,000 test reviews, and about a minute of loading: 69 minutes in total, or 5.8 hours for the five folds, run overnight. For comparison, the five DeBERTa folds took 2.4 hours and the five ELECTRA folds 0.8 hours. Two DeBERTa folds took 39 and 56 instead of 12–18 minutes, for reasons that were not identified.
- Reproducibility: every fold folder contains run_info.json (command, configuration, environment, data hashes, training history and, for Qwen3, the zero-shot and reload fingerprints), the full log, and the validation and test predictions. The LoRA adapters (about 175 MB per fold) and the DeBERTa checkpoints (1.7 GB per fold) are too large for the Moodle ZIP and are kept by the author. Without a GPU, the commands below regenerate the two Stage 4 files, A and B (Section 6.7), from the saved predictions in a few seconds. These scripts (and compare_runs.py for the pilot) were re-run on another computer (Linux, Python 3.10). They reproduced all 25,000 labels of every file uploaded in Stages 3 and 4, and files A and B matched byte for byte (SHA-256 17f55fc207e8… and 24bc9a952909…).

Reproduce Stages 3 and 4 with:

```
pip install -r requirements-llm.txt        # peft + accelerate, on top of requirements-gpu.txt
python code/run_cv.py --data-dir data --folds 5 --output runs/cv-deberta --name deberta
python code/run_cv.py --trainer llm --preset qwen3-8b --data-dir data --folds 5 \
    --output runs/cv-qwen3 --name qwen3 --eval-every 0.5          # about 5.8 h on a 24 GB GPU
# CPU only, a few seconds: pool the folds, combine, backfill
python code/assemble_cv.py --data-dir data --runs runs/cv-deberta --output ensemble-deberta
python code/assemble_cv.py --data-dir data --runs runs/cv-qwen3 --output ensemble-qwen3
python code/ensemble_cv.py --data-dir data --output ensemble-stage4 \
    --oof deberta=ensemble-deberta qwen3=ensemble-qwen3
python code/ensemble_cv.py --data-dir data --output ensemble-stage4-stack --choice stack \
    --oof deberta=ensemble-deberta qwen3=ensemble-qwen3
python code/apply_exact_match.py --train data/train.csv --test data/test.csv \
    --submission ensemble-stage4/submission.csv \
    --output ensemble-stage4/submission_exact.csv                              # file A
```

```
python code/apply_exact_match.py --train data/train.csv --test data/test.csv \
    --submission ensemble-stage4-stack/submission.csv \
    --output ensemble-stage4-stack/submission_exact.csv                    # file B
python code/analyze_stage34.py --data-dir data --output results/stage34   # report numbers
```

5.6 Stage 5 implementation

Environment and rule. Stage 5 ran on a rented machine (AutoDL cloud, China Northwest Zone B, C33 node with one NVIDIA RTX PRO 6000 Blackwell Server Edition 96 GB and a 50 GB data disk, ¥6.98 per hour including network). Environment: Python 3.12.3 on Linux, PyTorch 2.8.0+cu128, transformers 4.57.6, peft 0.18.1, accelerate 1.3.0. Qwen3-14B-Base is loaded in bfloat16 (26.3 GB on GPU); a memory probe peaks at 79.8 GB reserved and training at 85.2 GB, well below the 95 GB available. The five folds ran for 4.6 hours; the whole rental, including the model download and the smoke test, was about 5.4 hours (¥37). The folds run from code/run_stage5.sh. The copy in the submission, like those of the Stage 6 and 7 scripts, runs from the package root and leaves out the cloud-only step that zipped the results for download. The other operations (pooling the folds, computing combinations, backfilling, packing the results) used no GPU and took a few seconds. The selection rule was pre-registered: the Stage 5 combination would replace Qwen3-8B only if its OOF estimate exceeded 97.02%.

Run-time per 14B fold. About 31 minutes of training (about 3,330 tokens per second), 3.3 minutes for the half-epoch evaluation, 20 minutes to predict the 5,000 validation and 25,000 test reviews; 55 minutes total. This is 21% faster than the 8B model despite its 83% more parameters, because switching gradient checkpointing off has a stronger effect than the extra flops. The five folds and the combination took 4.6 hours in total.

Reproducibility. The saved predictions of all three families — DeBERTa, Qwen3-8B and Qwen3-14B — are in the submission. The commands below (CPU only, a few seconds) regenerate the final Stage 5 file from them:

```
python code/ensemble_cv.py --data-dir data --output ensemble-stage5 \
    --oof deberta=ensemble-deberta qwen3=ensemble-qwen3 qwen3_14b=ensemble-qwen3-14b
python code/apply_exact_match.py --train data/train.csv --test data/test.csv \
    --submission ensemble-stage5/submission.csv --output ensemble-stage5/submission_exact.csv
```

The 14B adapters (1.3 GB) are too large for the Moodle ZIP and are kept by the author.

5.7 Stage 6 implementation

Environment and cost. Stage 6 ran on another AutoDL machine with one NVIDIA H800 PCIe (79.2 GB usable) and a 150 GB data disk. Software: Python 3.12.3 on Linux, PyTorch 2.8.0+cu128, transformers 4.57.6 and peft 0.18.1, the same versions as Stage 5. The only code change is a new preset, mistral-24b, in code/finetune_llm.py; the 132 unit tests (two more than in Stage 5) cover it. Weights take 43.0 GB on the GPU. The memory probe peaked at 48.4 GB reserved and training at 54.8 GB. A 3-minute smoke test on a few hundred reviews ran the whole pipeline end to end before the folds started. The folds run from code/run_stage6.sh.

Run-time per fold. About 59 minutes of training (about 1,750 tokens per second), 4.2 minutes for the half-epoch evaluation and 25 minutes to predict the 5,000 validation and 25,000 test reviews: 89 minutes per fold, 7.4 hours for the five folds that were kept, plus 1.5 hours for the diverged first run of fold 3.

Divergence guard. finetune_llm.py refuses to save a fold whose final validation accuracy is below 60%: it writes no run_info.json and marks the adapter incomplete, so run_cv.py stops and a re-run trains the fold again. This guard caught the fold-3 failure described in Section 6.9. The full log of the failed run is kept in the fold's train_log.txt, ahead of the successful one.

Reproducibility. The saved predictions of all four families are in the submission. Without a GPU, the commands below rebuild file D from them in a few seconds; re-run on Linux (Python 3.10, CPU only) they reproduced it byte for byte (SHA-256 4efa15e47644…). The adapters (1.8 GB for the five folds) are too large for the Moodle ZIP and are kept by the author.

```
python code/assemble_cv.py --data-dir data --runs runs/cv-mistral-24b \
    --output ensemble-mistral-24b
python code/ensemble_cv.py --data-dir data --output ensemble-stage6 \
    --oof deberta=ensemble-deberta qwen3=ensemble-qwen3 qwen3_14b=ensemble-qwen3-14b \
          mistral_24b=ensemble-mistral-24b
python code/apply_exact_match.py --train data/train.csv --test data/test.csv \
    --submission ensemble-stage6/submission.csv \
    --output ensemble-stage6/submission_exact.csv                          # final D
```

5.8 Stage 7 implementation

Environment. Stage 7 ran on an AutoDL machine with one NVIDIA RTX PRO 6000 Blackwell Server Edition (95 GB). The model needs a newer software stack than Stages 5–6: Python 3.12.3, PyTorch 2.8.0+cu128, transformers 5.22.0 and peft 0.21.2, plus flash-linear-attention 0.4.2, which provides fast GPU kernels for the DeltaNet layers. Version 0.4.2 is pinned because newer releases need a newer Triton than PyTorch 2.8 ships (requirements-stage7.txt). The optional causal-conv1d kernel was not installed, so the short convolution inside each DeltaNet layer used the plain PyTorch version; this affects speed, not results. The new code is the qwen3.8-27b preset in code/finetune_llm.py, which loads the vision-language checkpoint, picks out its text part and places LoRA on its linear layers. The 141 unit tests (nine more than in Stage 6) cover the name check, the layer selection and the preset. A script printed the model's layer types, the LoRA targets and the answer tokens on the CPU before any GPU time was used, and a 7-minute smoke test ran the whole pipeline end to end. The folds run from code/run_stage7.sh.

Memory and run-time per fold. The memory probe peaked at 54.0 GB reserved and training at 56.3 GB. Training took about 88 minutes (about 1,160 tokens per second), the half-epoch evaluation 6.9 minutes and prediction of the 5,000 validation and 25,000 test reviews 40 minutes: 136 minutes per fold and 11.4 hours for the five folds. The smoke test had estimated 206 minutes per fold; its prediction-time estimate from a few hundred reviews was too high. No fold diverged and no update was skipped.

Reproducibility. The saved predictions of all five families are in the submission. Without a GPU, the commands below rebuild file E from them in a few seconds; re-run on Linux (CPU only) they reproduced it byte for byte (SHA-256 9ac9d596470d…). The adapters are too large for the Moodle ZIP and are kept by the author.

```
python code/assemble_cv.py --data-dir data --runs runs/cv-qwen38-27b \
    --output ensemble-qwen38-27b
python code/ensemble_cv.py --data-dir data --output ensemble-stage7 \
    --oof deberta=ensemble-deberta qwen3=ensemble-qwen3 qwen3_14b=ensemble-qwen3-14b \
          mistral_24b=ensemble-mistral-24b qwen38_27b=ensemble-qwen38-27b
python code/apply_exact_match.py --train data/train.csv --test data/test.csv \
    --submission ensemble-stage7/submission.csv \
    --output ensemble-stage7/submission_exact.csv                          # final E
```

6. Results and Observations

6.1 Model comparison

Table 1 summarises the best configuration of every family, and Table 2 shows how each family responds to the n-gram order. Figure 5 visualises Table 1 with ± one standard deviation across folds.

Table 1. Best configuration per model family, stratified 5-fold CV on the 25,000 training reviews.

Model family	Best configuration	CV accuracy	± std
Multinomial naive Bayes (raw counts)	(counts, uni+bi+tri, alpha=1.0)	88.63%	0.72%
Multinomial naive Bayes (binary presence)	(binary, uni+bi+tri, alpha=1.0)	89.20%	0.78%
TF-IDF + L2 logistic regression	(uni+bi+tri, C=16.0)	91.30%	0.37%
NB-LR (NB log-count ratios + logistic regression)	(uni+bi+tri, C=1.0, beta=0.75)	91.78%	0.35%

Table 2. Effect of the maximum n-gram order (best C/β for each cell, CV accuracy).

Max n-gram order	NB (counts)	NB (binary)	TF-IDF + LR	NB-LR
1 (unigrams)	0.8406	0.8517	0.8992	0.8978
2 (+ bigrams)	0.8750	0.8811	0.9130	0.9166
3 (+ trigrams)	0.8863	0.8920	0.9130	0.9178

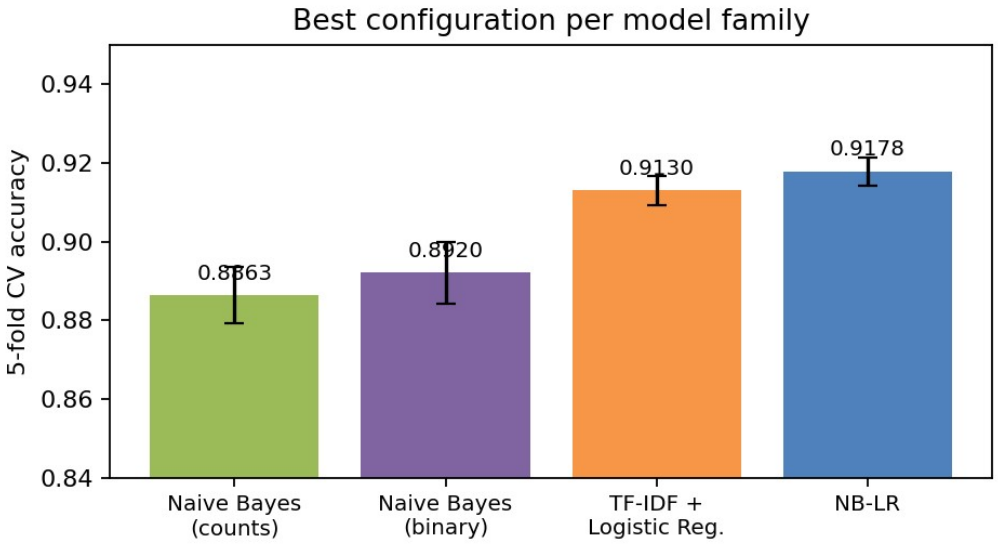


Figure 5. Best configuration per family; error bars are ± 1 standard deviation over the five folds.

Observations:

- Adding bigrams is the single most valuable change for every family (+2.1 to +3.4 points over unigrams); trigrams add a further 0.1–0.5 points and are most useful for the two NB-based models, which cannot otherwise model interactions between words.
- Binarising counts helps naive Bayes by about 1 point at every n-gram order, consistent with Pang, Lee & Vaithyanathan (2002) on movie reviews and with Metsis et al. (2006) on spam filtering — for sentiment, the presence of a strong phrase matters more than how many times it is repeated.
- The discriminative models beat naive Bayes by 2–3 points, and NB-LR beats TF-IDF LR by ~0.5 points at every n-gram order. Scaling features by the NB log-count ratio gives LR a far better starting geometry and allows much stronger regularisation ($C \approx 1$ instead of 16–64) without under-fitting.
- NB-LR is remarkably insensitive to its hyper-parameters (Figure 6): all nine (C, β) settings with trigrams lie within 0.3 points, and $\beta = 0.5$–0.75 is consistently best, confirming that the interpolation towards the NB solution is a useful regulariser. The TF-IDF LR plateau at $C \geq 16$ has

the same message — this problem wants weak regularisation of a discriminative model on rich features.

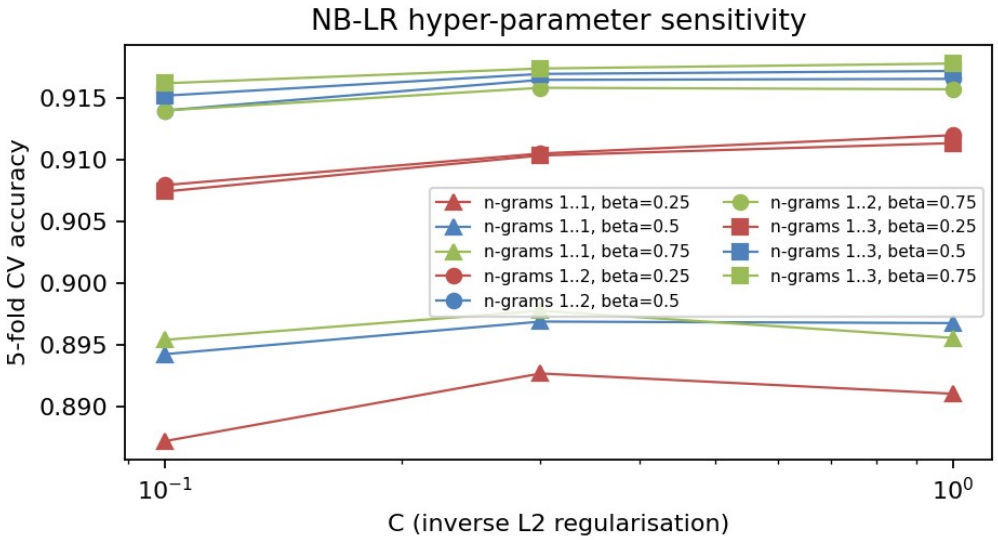


Figure 6. NB-LR sensitivity to C and β for the three n-gram orders.

6.2 Selected model and detailed evaluation

The selected model is NB-LR (uni+bi+tri, C=1.0, beta=0.75): 91.78% ± 0.35% CV accuracy (fold accuracies 91.3%, 91.9%, 92.3%, 91.7%, 91.7%). Its out-of-fold confusion matrix (Figure 7) is nearly symmetric: precision 90.7%, recall 93.2% and F1 91.9% for the positive class, specificity 90.4%. The model is slightly more willing to call a review positive (1,199 false positives vs 856 false negatives), which matches the 51.1% positive share it predicts on the test set.

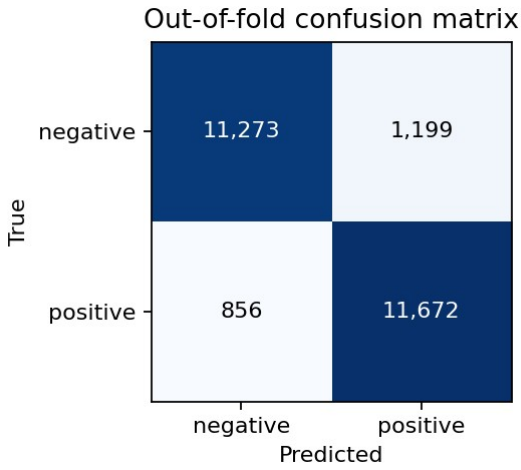


Figure 7. Out-of-fold confusion matrix of the selected NB-LR model (25,000 training reviews).

Figure 8 (left) shows accuracy is stable across review lengths (91.1%–92.6%); the longest reviews (500+ words) are actually the easiest, because they contain more sentiment-bearing phrases to vote. Figure 8 (right) shows the decision score is a well-calibrated confidence: the 20% of reviews with the smallest |score| are classified at only 71.1%, whereas the top 60% are above 97.6% and the top quintile reaches 99.6%. In other words, 70% of all errors are concentrated in the least-confident fifth of the predictions.

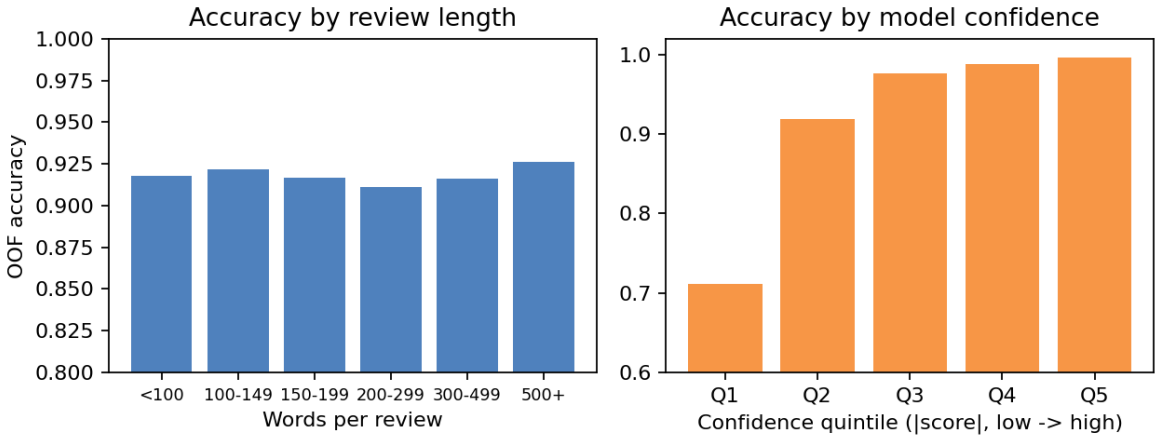


Figure 8. Out-of-fold accuracy by review length (left) and by quintile of |decision score| (right).

The learning curve (Figure 9a) rises from 85.8% with 999 reviews to 91.3% with 19,999, still climbing by ≈ 0.2–0.6 points per doubling, while training accuracy is 100% throughout: with ~900 k features the model can memorise any training set, so performance is limited by data and by the representation rather than by optimisation. Figure 9b shows the most influential features of the final model: numeric scores ("4 10", "7 10", "9 10"), negated and contrastive phrases ("not funny", "potential but", "not worth the") and strong evaluative words ("worst", "gem", "highly recommended"). These are exactly the cues a human would use, which gives confidence that the model is not exploiting artefacts.

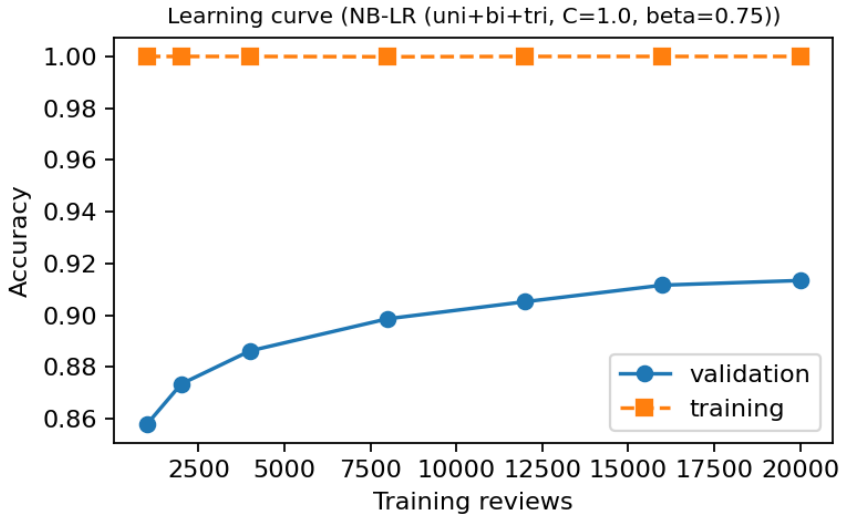


Figure 9a. Learning curve of the selected model on fold 1 (validation = 5,001 reviews).

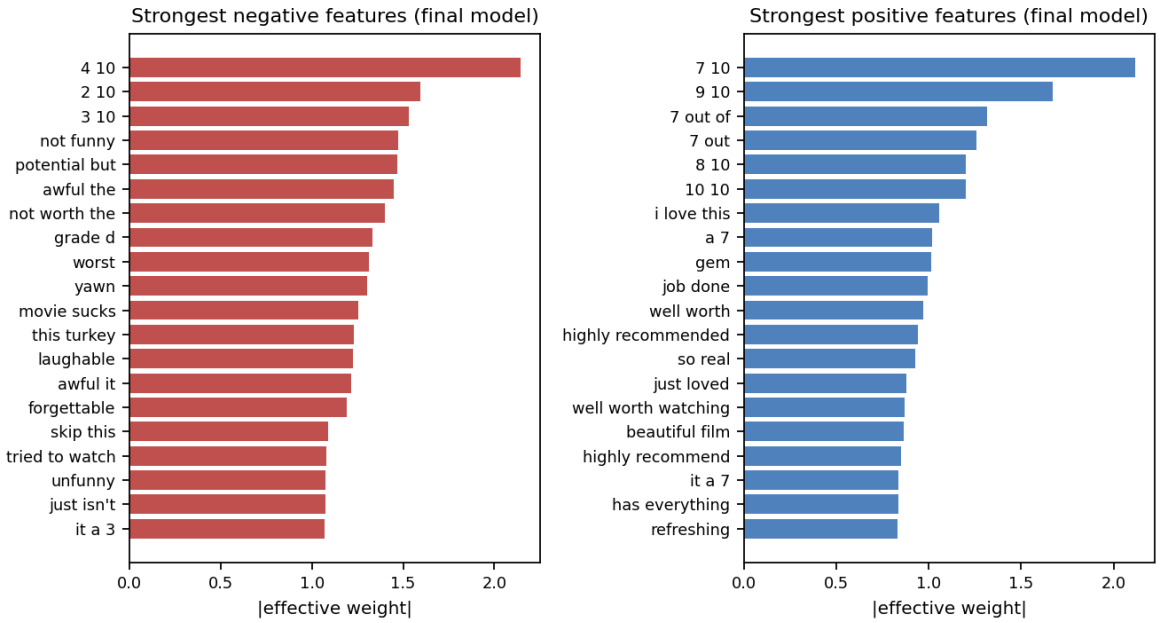


Figure 9b. Strongest negative and positive features of the final model (effective weight w·r on the presence feature).

6.3 Stage 2: fine-tuned transformer

Table 3. NB-LR and DeBERTa-v3-large on the same 2,500 hold-out reviews (NB-LR: out-of-fold scores; CI: 95% Wilson interval).

Model	Accuracy	Precision (pos.)	Recall (pos.)	F1 (pos.)	Errors
NB-LR (Stage 1)	91.76%	90.18%	93.77%	91.94%	206
DeBERTa-v3-large	96.72% (CI 95.9–97.3)	97.18%	96.25%	96.71%	82
Mean of both (CV inside hold-out)	96.82%	–	–	–	–
Stacked LR (CV inside hold-out)	96.86%	–	–	–	–

Figure 10 (left) shows the hold-out accuracy during fine-tuning. After only half an epoch it is 95.60%, already 3.8 points above NB-LR; it peaks at 96.72% after 1.5 epochs and falls slightly to 96.48% at the end, so the checkpoint from epoch 1.5 was kept. Picking the best of 5 evaluations makes the hold-out figure slightly optimistic, which is why the public leaderboard (Section 6.11) is used as an independent check. The improvement over NB-LR is far beyond noise: on the 192 hold-out reviews where exactly one model is right, DeBERTa is right 158 times and NB-LR 34 times (exact McNemar test, $p < 10^{-19}$; Figure 11). Unlike NB-LR, DeBERTa also has balanced errors (35 false positives vs 47 false negatives).

Figure 10 (right) breaks the hold-out down by length. NB-LR varies between 89.8% and 93.3% across the bins (on all 25,000 out-of-fold predictions, Figure 8, the variation is smaller), while DeBERTa stays between 96.5% and 97.2% in every bin — including reviews over 512 words, which are always truncated. This indicates that the head+tail window keeps the part of the review that carries the verdict. As for NB-LR, the size of the transformer's score is a good confidence measure: 68 of its 82 errors (83%) fall in the least-confident fifth of hold-out reviews, where accuracy is 86.2%; in the other four fifths it is 98.3%–99.8%.

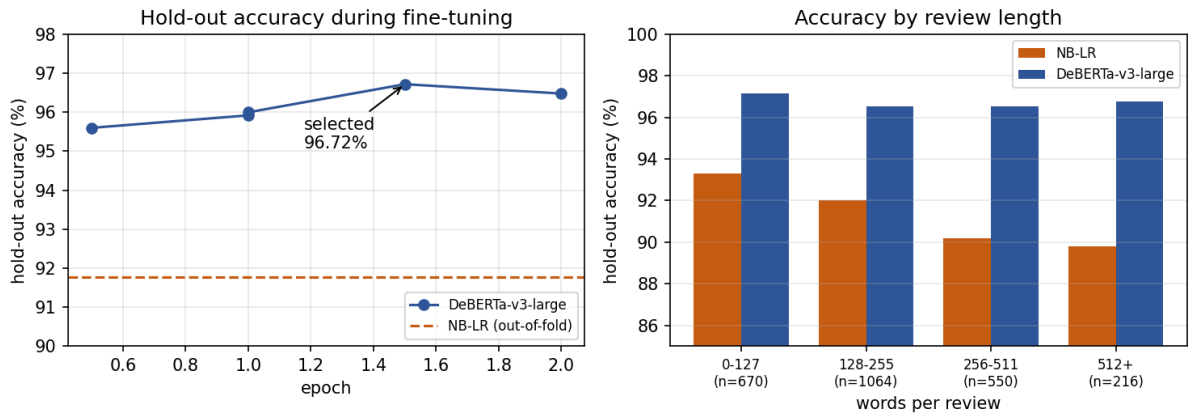


Figure 10. DeBERTa-v3-large hold-out accuracy during fine-tuning (left) and hold-out accuracy of both models by review length (right).

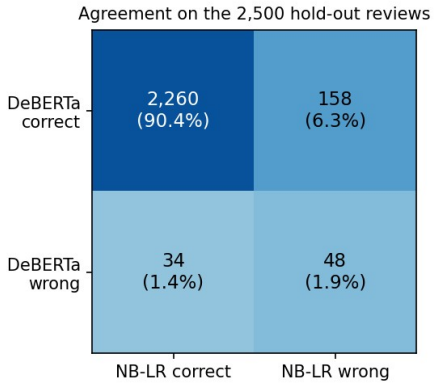


Figure 11. Agreement of the two models on the 2,500 hold-out reviews.

Combining DeBERTa with NB-LR helps only a little. Averaging the standardised scores is estimated at 96.82% and stacking at 96.86%, against 96.72% for DeBERTa alone. The stack's 0.03-point edge over the mean is below the 0.05-point threshold, so the simpler mean was selected. The reason the gain is small is visible in Figure 11: NB-LR is right on only 34 reviews that DeBERTa gets wrong, and 48 of DeBERTa's 82 errors are shared with NB-LR.

6.4 Stage 2: several runs and the final ensemble

Table 4. All Stage 2 runs on the same 2,500 hold-out reviews (best of five evaluations during training; 95% Wilson interval), and the mean ensembles (CV inside the hold-out).

Model	Best epoch	Train min	Hold-out accuracy (95% CI)	Errors
DeBERTa-v3-large, seed 42	1.5	13.1	96.72% (95.9–97.3)	82
DeBERTa-v3-large, seed 1	2.0	21.5	96.72% (95.9–97.3)	82
DeBERTa-v3-large, seed 2	1.5	11.2	96.20% (95.4–96.9)	95
RoBERTa-large, seed 42	2.0	6.2	96.24% (95.4–96.9)	94
NB-LR (Stage 1, out-of-fold)	–	–	91.76% (90.6–92.8)	206
Mean: DeBERTa seed 42 + NB-LR	–	–	96.82%	–
Mean: all five models (final)	–	–	96.90% (stack 96.87%)	77

Seed variation. The three DeBERTa runs differ only in their random seed, yet their hold-out accuracy ranges from 96.20% to 96.72% — a spread of 0.5 points, almost as wide as the confidence interval of one run. The first run (seed 42) happened to be at the top of this range, so its single-model figure in Section

6.3 is somewhat lucky. RoBERTa-large (96.24%) is as good as an average DeBERTa run and needed less than half the training time. One small detail: for the seed-2 run the hold-out accuracy measured during training (96.28%) and after reloading the saved checkpoint (96.20%) differ by two reviews. The two evaluations use different batch sizes, so the padding and the bfloat16 rounding differ, which can flip reviews whose margin is almost exactly zero. All tables use the reloaded checkpoint, because that is the model that made the test predictions.

Why the ensemble gains little. Figure 12 (right) shows that the four transformer runs produce almost the same scores (correlations 0.98–0.99), even RoBERTa, which has a different tokenizer and pre-training. Their errors overlap accordingly: 49 hold-out reviews are misclassified by all four runs, and 36 of them by NB-LR as well — these are mostly the label-noise and irony cases of Section 7.2, which no model in the ensemble can fix. NB-LR is much less correlated (r = 0.79–0.81), but it is also five points less accurate. The mean of all five models is estimated at 96.90%; stacking (96.87%) is not better, so the mean was used. Compared with the best single run, the ensemble corrects 17 hold-out reviews and introduces 12 new errors; this difference is not statistically significant (exact McNemar test, p = 0.46). Its value lies more in being less dependent on one lucky seed than in a measurable accuracy gain.

Ablation. For a descriptive check, the mean combiner was also evaluated for all 26 subsets of two or more models (results/stage2/ensemble2_summary.json). They range from 96.14% to 97.00%. The four transformers without NB-LR score 97.00% and the three DeBERTa runs alone 96.60%. The best subset is only 2 reviews better than the full ensemble, far below the noise level. Choosing it would therefore mean selecting on chance, so the pre-specified ensemble of all models was kept. On the test set the final ensemble changes the label of 290 reviews (1.2%) relative to DeBERTa seed 42, and of 246 relative to the earlier DeBERTa + NB-LR ensemble.

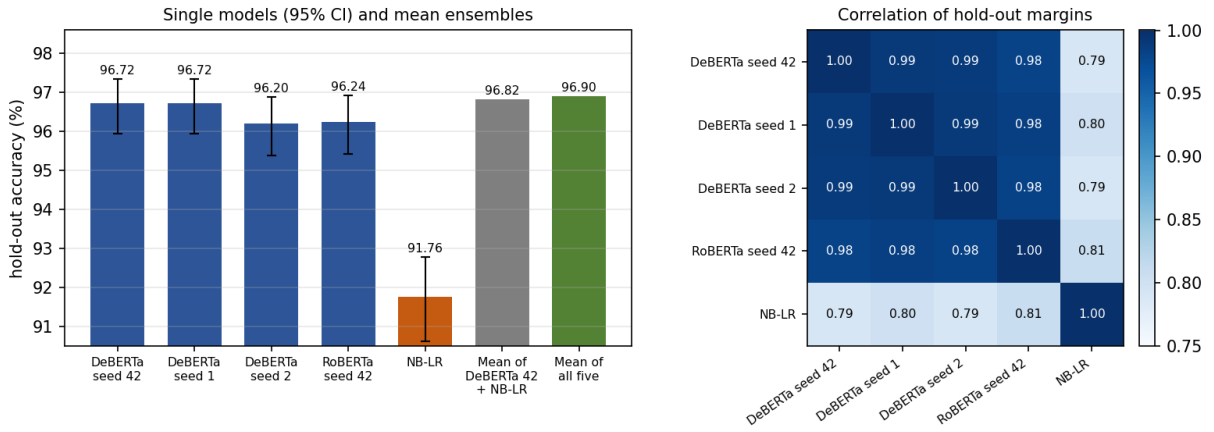


Figure 12. Hold-out accuracy of every Stage 2 model and of the two mean ensembles (left), and correlation of the models' hold-out scores (right).

6.5 Stage 3: cross-validated transformers

Table 5. Model families on the same 5-fold partition: out-of-fold accuracy over the 25,000 training reviews (95% Wilson interval), spread over the five folds, GPU time, and the public score of the averaged fold models.

Model family	OOF accuracy (95% CI)	Folds	GPU time	Public
NB-LR (Stage 1; its own CV folds)	91.78% (91.43–92.11%)	91.3–92.3%	CPU	0.91730
DeBERTa-v3-large (Stage 2 recipe)	96.73% (96.50–96.94%)	96.4–97.1%	2.4 h	0.97150
ELECTRA-large (same recipe)	96.41% (96.17–96.64%)	95.9–97.0%	0.8 h	0.96940
DeBERTa-v3-large, fresh re-run	96.77% (96.54–96.98%)	96.5–97.1%	1.8 h	0.97010 *
Qwen3-8B-Base + LoRA (Stage 4)	97.02% (96.81–97.23%)	96.8–97.4%	5.8 h	0.97360

** with the exact-match backfill of Section 4.10.*

The Stage 2 DeBERTa recipe reaches 96.73% over all 25,000 OOF reviews, which confirms that the Stage 2 hold-out figure (96.72%) was not inflated. On the 2,500 reviews the two share, the fold-0 model scores 96.64% and the Stage 2 model 96.72%; they disagree on only 40 reviews, of which the Stage 2 model gets 21 and the fold-0 model 19 right (McNemar p = 0.87). For one and the same recipe the five folds lie between 96.42% and 97.10% (standard deviation 0.26 points), which shows directly how far a single 5,000-review validation set can move by chance.

The average of the five fold models scored 0.97150 on the public leaderboard. That is 0.27 points above the single Stage 2 DeBERTa and 0.10 above the five-model Stage 2 ensemble, although each fold model saw fewer reviews (20,000) than a Stage 2 run. The fold models agree on 97.1% of the test reviews (margin correlations 0.991–0.994). The gain therefore comes from the remaining 2.9%, where averaging five models that together saw every label helps. Neither a third encoder nor more tuning helped:

- **ELECTRA-large** reaches 96.41%, significantly below DeBERTa: DeBERTa is right on 263 reviews that ELECTRA misses, and ELECTRA on 184 the other way (p = 0.0002). With a margin correlation of 0.984, the combination rule kept DeBERTa alone: the stack would have gained 0.05 points, less than one standard error. ELECTRA ran with DeBERTa's hyper-parameters, so this result concerns this recipe, not ELECTRA in general.
- **The fresh re-run** first screened four settings on the 2,500-review hold-out. They scored 96.3–96.8%, a spread this hold-out cannot resolve, so the Stage 2 setting (learning rate 10^{-5}, 2 epochs) was kept. Its update-0 accuracy of 49.9% confirms that every fold started from the general checkpoint with a random head. The re-run's OOF accuracy (96.77%) cannot be distinguished from the first run's: 120 against 130 discordant reviews, p = 0.57, margin correlation 0.994. Its backfilled file nevertheless scored 0.97010, 0.14 below the first run, although the two files differ on only 110 of the 25,000 test reviews. Two runs of one recipe can thus differ by about 14 public reviews by chance (Section 7.4).
- **Self-training** (Xie et al., 2020). The four Stage 2 transformers agreed on 24,159 test reviews. The most confident 70% of each class (16,911 reviews) were added to the training data with weight 0.5, and DeBERTa was trained for one more epoch at learning rate 5×10^{-6}. An implementation error, found afterwards, made the run start from the already fine-tuned Stage 2 weights instead of the general checkpoint. It therefore tested continued training on the model's own confident answers, which carry almost no gradient signal. Hold-out accuracy fell from 96.72% to 95.84% after half an epoch and recovered to 96.52%. The best checkpoint was the initial one, so the output was identical to the Stage 2 model. The intended experiment, a new model trained from the general checkpoint on the enlarged data, was not run, so this result says nothing about it; the idea was not pursued.

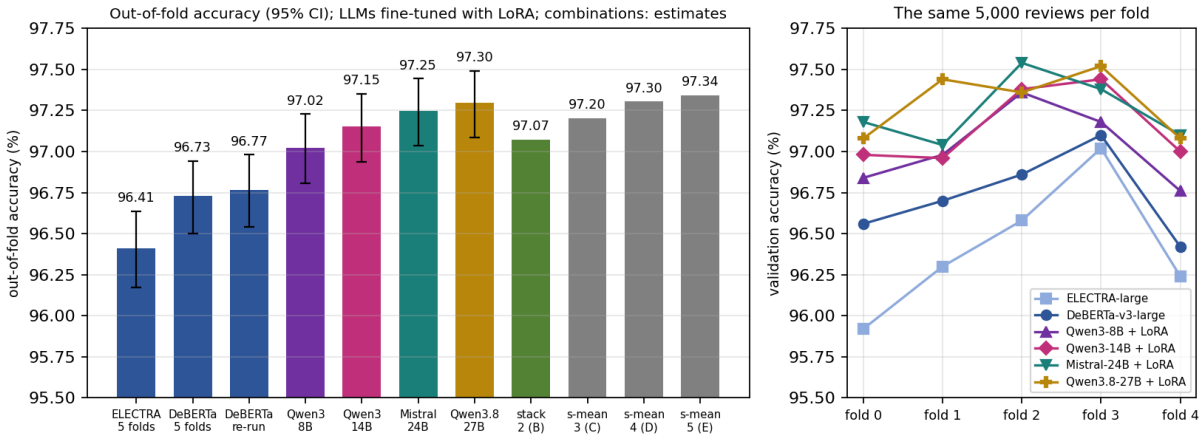


Figure 13. Out-of-fold accuracy of every 5-fold model family with 95% intervals, and the estimates of the combinations B–E of Stages 4–7 (left); validation accuracy fold by fold (right).

6.6 Stage 4: Qwen3-8B-Base with LoRA

Table 6. Qwen3-8B-Base + LoRA fold by fold, and DeBERTa-v3-large on the same validation reviews. Zero-shot: the untrained classifier on 256 validation reviews; the other columns use all 5,000.

Fold	Zero-shot	Half epoch	Final	DeBERTa	Only Qwen3 / only DeBERTa right	McNemar p
0	95.31%	96.42%	96.84%	96.56%	57 / 43	0.19
1	93.75%	96.66%	96.98%	96.70%	59 / 45	0.20
2	94.14%	97.18%	97.36%	96.86%	66 / 41	0.02
3	94.53%	96.82%	97.18%	97.10%	49 / 45	0.76
4	95.70%	96.88%	96.76%	96.42%	67 / 50	0.14
All 25,000	94.69% †	96.79%	97.02%	96.73%	298 / 224	0.0014

† mean over the 5 × 256 checked reviews; the half-epoch value is the mean of the five folds.

Before any training, the classifier initialised from the model's own answer is already 93.8–95.7% accurate (AUC 0.980–0.995). That is well above NB-LR, and about what DeBERTa reaches after half an epoch of fine-tuning. One epoch of LoRA fine-tuning removes almost half of the remaining errors: 96.79% on average at half an epoch and 97.02% at the end. The end is higher in four of the five folds, so one epoch was not obviously too long; longer training was not tried.

Over all 25,000 OOF reviews, Qwen3 is 97.02% accurate (95% CI 96.81–97.23%) against 96.73% for DeBERTa. It is right on 298 reviews that DeBERTa gets wrong and wrong on 224 that DeBERTa gets right: a net 74 reviews, exact McNemar p = 0.0014. It is better in all five folds, but only in fold 2 would the difference have been significant on its own (p = 0.02). The pilot on fold 0 alone (57 against 43, p = 0.19) could not have established it — the argument of Section 4.7 in practice. The comparison also slightly favours DeBERTa, whose checkpoints were selected on the validation folds; with its final-epoch weights DeBERTa averages about 96.65%.

Figure 14 shows where the gain comes from. On reviews under 200 words (59% of the data) Qwen3 is only 0.17 points better (p = 0.17). On reviews of 200 words or more it is 0.48 points better (132 against 83, p = 0.001), and the advantage grows with length to 0.65 points above 500 words (98.09% vs 97.44%). Two thirds of the net gain (49 of 74 reviews) comes from these longer 41% of the reviews. Two explanations fit. The 1,024-token window keeps 98% of the reviews whole, whereas DeBERTa truncates 12%. And long reviews contain more plot description, digressions and comparisons, where knowledge about films helps to tell description from verdict (Section 7.3).

Qwen3's errors are balanced (367 false positives, 377 false negatives; 49.96% predicted positive on the test set). As for the other models, they are concentrated where it is unsure: 89% fall in the least-confident fifth of the reviews, and outside it accuracy is at least 99.06%. The hypothesis of Section 4.8 is confirmed only in part. Qwen3 makes fewer errors, but not very different ones: its margins correlate with DeBERTa's at 0.983, hardly less than ELECTRA's 0.984, and 520 of its 744 errors (70%) are also DeBERTa errors (Figure 14, right).

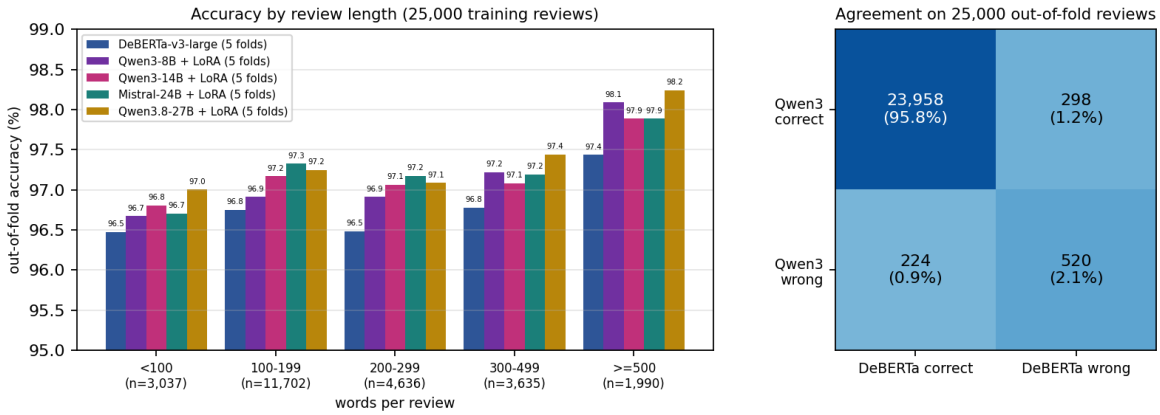


Figure 14. Out-of-fold accuracy of DeBERTa and the four language models by review length (left), and the agreement of Qwen3-8B and DeBERTa on the 25,000 out-of-fold reviews (right).

6.7 Stage 4: combining Qwen3 with DeBERTa

Table 7. Combinations of DeBERTa and Qwen3 on the 25,000 OOF rows (estimated accuracy; the stack by 10 × repeated 10-fold CV), paired with Qwen3 alone on the same reviews.

Combination	Estimate	Only combination / only Qwen3 right	McNemar p	Role
Qwen3 alone	97.024%	–	–	rule choice → file A
DeBERTa alone	96.728%	224 / 298	0.0014	
s-mean	97.012%	177 / 180	0.92	not better
stack (logistic regression)	97.074%	105 / 92 ‡	0.39	gain < 1 SE → file B
z-mean (comparison only)	97.025%	182 / 182	1.00	
mean of raw margins (comparison only)	97.020%	176 / 177	1.00	
rank-mean (comparison only)	97.112%	140 / 118	0.19	

‡ the stack fitted on all 25,000 rows (in-sample accuracy 97.076%).

No combination is clearly better than Qwen3 alone (Table 7, Figure 13). s-mean is 0.012 points worse: it fixes 177 of Qwen3's errors but breaks 180 of its correct answers, because the weaker DeBERTa receives the same weight. The stack learns this by itself. Per standard deviation of its input it weights Qwen3 2.8 times as heavily as DeBERTa (bias 0.06), and gains an estimated 0.05 points — less than the 0.11-point standard error that the rule requires. The rule therefore chose Qwen3 alone. Rank-mean has the highest estimate (97.11%), but it was excluded in advance and its gain is not significant (p = 0.19); choosing it after seeing this table would be selecting on noise. On the test set the stack changes 119 of Qwen3's 25,000 labels, which made it a useful second final selection; it kept that place until Stage 7 (Section 6.11).

6.8 Stage 5: Qwen3-14B-Base and the three-model average

Table 8. Qwen3-14B-Base + LoRA fold by fold, against Qwen3-8B-Base on the same 5,000 validation reviews of each fold (identical recipe).

Fold	Zero-shot	Half epoch	14B final	8B final	Only 14B / only 8B right	McNemar p
0	95.70%	96.78%	96.98%	96.84%	36 / 29	0.46
1	94.53%	96.38%	96.96%	96.98%	30 / 31	1.00
2	95.31%	97.16%	97.38%	97.36%	27 / 26	1.00
3	96.09%	97.16%	97.44%	97.18%	34 / 21	0.10
4	94.53%	97.12%	97.00%	96.76%	36 / 24	0.15
All 25,000	95.23% †	96.92%	97.15%	97.02%	163 / 131	0.070

† mean over the 5 × 256 checked reviews.

The larger model is better, but by little. Zero-shot it reaches 94.5–96.1% (8B: 93.8–95.7%), and after one epoch 97.15% OOF (95% CI 96.94–97.35%), 0.13 points above the 8B model. It wins in four of the five folds, and over all reviews it is right on 163 that the 8B model gets wrong against 131 the other way: a net 32 reviews, McNemar p = 0.07. That is suggestive but not significant at 5%. The gain was predicted before the run at +0.05 to +0.2 points, and it lies inside that range. Against DeBERTa, the 14B model's lead is clearly significant (302 against 196, p = 2.3e-6).

Where the gain comes from differs from Stage 4 (Figure 14, left). The 14B model is 0.26 points better on reviews of 100–199 words, the largest group, but 0.14 and 0.20 points worse on reviews of 300–499 and 500+ words. None of these per-length differences is significant on its own. They suggest, however, that the extra capacity helps on short reviews, where a few phrases decide the label, rather than on long ones, which the 8B model already reads in full. The two Qwen3 models agree on 98.82% of the OOF reviews and share 581 errors, so a larger model of the same family adds little new information. The margins correlate at 0.992, the highest of any pair.

Table 9. Combinations of DeBERTa, Qwen3-8B and Qwen3-14B on the 25,000 OOF rows (estimate), paired with Qwen3-14B alone.

Combination	Estimate	Only combination / only 14B right	McNemar p	Role
Qwen3-14B alone	97.152%	–	–	
s-mean	97.204%	131 / 118	0.45	rule choice → file C
stack (logistic regression)	97.221%	–	–	gain over s-mean < 1 SE
z-mean (comparison only)	97.148%	129 / 130	1.00	
mean of raw margins (comparison only)	97.200%	131 / 119	0.49	
rank-mean (comparison only)	97.200%	108 / 96	0.44	

With three families the rule of Section 4.7 chose s-mean, the plain average of the standardised margins (97.204%). The stack estimate was 0.017 points higher, less than the 0.10-point standard error, so the simpler combiner was kept. The stack would weight the 14B model 2.4 times as heavily as the 8B model, and the 8B model 1.7 times as heavily as DeBERTa. Unlike in Stage 4, s-mean now helps: with two Qwen3 models out of three, the weaker DeBERTa has only a third of the vote. It still fixes only 13 more of the 14B model's errors than it breaks (p = 0.45).

Since 97.20% is above the pre-registered threshold of 97.02%, the rule replaced the Stage 4 rule choice (file A) with this average plus backfill (file C). Compared with the OOF predictions of the Stage 4 stack

cross-fitted once on the shared folds (file B, 97.08% correct; the 97.074% in Table 7 averages repeated random splits), file C is right on 74 OOF reviews where B is wrong and wrong on 43 where B is right (p = 0.005). Against Qwen3-8B alone (file A) it is 163 against 118 (p = 0.009). Both differences are significant, so Stage 5 is the first stage since Stage 4's Qwen3 whose improvement is measurable on the OOF rows. The improvement remains small: on the test set C differs from B on 73 reviews and from A on 174.

6.9 Stage 6: Mistral-Small-24B-Base and the four-model average

Table 10. Mistral-Small-24B-Base + LoRA fold by fold, against Qwen3-14B-Base on the same 5,000 validation reviews of each fold (identical recipe). Fold 3 is the seed-43 re-run.

Fold	Zero-shot	Half epoch	24B final	14B final	Only 24B / only 14B right	McNemar p
0	95.31%	96.46%	97.18%	96.98%	38 / 28	0.27
1	94.92%	92.34%	97.04%	96.96%	34 / 30	0.71
2	95.31%	96.54%	97.54%	97.38%	36 / 28	0.38
3 ‡	95.70%	96.82%	97.38%	97.44%	30 / 33	0.80
4	96.88%	95.78%	97.10%	97.00%	40 / 35	0.64
All 25,000	95.63% †	95.59%	97.25%	97.15%	178 / 154	0.21

† mean over the 5 × 256 checked reviews. ‡ training seed 43; the seed-42 run of this fold diverged (see below).

One failed run. The first run of fold 3 (training seed 42) diverged. It started normally (zero-shot 95.70% on the check reviews), but its training loss jumped to 0.88–1.03 between updates 275 and 400 and then settled at 0.69, which is ln 2: the model had stopped telling the classes apart. The half-epoch check found 49.90% validation accuracy, and the final model 52.08%, predicting only 15% of the test reviews as positive. No loss value was infinite and memory use was normal, so this was an unstable optimisation, not a software fault. The divergence guard (Section 5.7) refused to save the fold. It was retrained once, with training seed 43 and nothing else changed; the fold split, which depends on the separate split seed, stayed the same. The re-run was normal from the start (96.82% at half an epoch, 97.38% at the end). Changing the seed of one fold after a failure is a deviation from the plan. It is reported here because it affects only the training randomness, not the data or the recipe, and because the failed result was never used.

Mistral-24B is the best single model of the project, but only slightly. Zero-shot it reaches 94.9–96.9% on the 256 checked reviews, and after one epoch 97.25% OOF (95% CI 97.04–97.44%), 0.10 points above Qwen3-14B. It wins in four of the five folds; over all reviews it is right on 178 that the 14B model gets wrong, against 154 the other way (p = 0.21, not significant). It is significantly better than Qwen3-8B (224 against 168, p = 0.005) and DeBERTa (316 against 186, p = 7.1e-9). It is the first model whose errors are exactly balanced (345 false positives, 343 false negatives). Its half-epoch accuracy varied more than that of the Qwen3 models (92.3–96.8%), but every fold ended at 97.0% or higher.

The model was chosen because it was trained on different data, so the key question is whether it errs differently. Partly. Its margins correlate with Qwen3-14B at 0.988, a little less than the two Qwen3 models with each other (0.992), and the two models agree on 98.67% of the OOF reviews (Qwen3 pair: 98.82%). Both are wrong on 534 reviews, against 581 for the two Qwen3 models. By review length (Figure 14) it follows the 14B model: best on reviews of 100–199 words (97.33%) and no better than Qwen3-8B on the longest ones.

Table 11. Combinations of DeBERTa, Qwen3-8B, Qwen3-14B and Mistral-24B on the 25,000 OOF rows (estimate), paired with Mistral-24B alone.

Combination	Estimate	Only combination / only 24B right	McNemar p	Role
Mistral-24B alone	97.248%	–	–	
s-mean	97.304%	137 / 123	0.42	rule choice → file D
stack (logistic regression)	97.350%	–	–	gain over s-mean < 1 SE
z-mean (comparison only)	97.324%	139 / 120	0.26	
mean of raw margins (comparison only)	97.292%	130 / 119	0.53	
rank-mean (comparison only)	97.256%	126 / 124	0.95	

The rule chose s-mean again (97.304%); the stack was 0.046 points higher, less than the 0.10-point standard error. Against Mistral-24B alone, s-mean fixes 137 errors and breaks 123 correct answers (p = 0.42): the four-model average is not significantly better than its best member. Its estimate is above the pre-registered threshold of 97.254%, so the rule replaced C with this average plus backfill (file D).

The more relevant comparison is with the files it replaces and keeps. On the OOF rows, D is right on 44 reviews where C is wrong and wrong on 19 where C is right (p = 0.002); against B it is 94 against 38 (p = 1.2e-6). Both differences are significant, but D's lead over C is only 0.10 points: D and C agree on 99.75% of the OOF reviews and differ on 42 test reviews. Stage 6 therefore confirms the pattern of Stage 5: a fourth large model helps the average a little, measurably on the OOF rows, but less than its own single-model gain would suggest, because all three language models already share most of their errors.

6.10 Stage 7: Qwen3.8-27B and the five-model average

Table 12. Qwen3.8-27B + LoRA fold by fold, against Mistral-24B on the same 5,000 validation reviews of each fold (identical recipe).

Fold	Zero-shot	Half epoch	27B final	24B final	Only 27B / only 24B right	McNemar p
0	87.11%	95.90%	97.08%	97.18%	27 / 32	0.60
1	88.28%	97.16%	97.44%	97.04%	41 / 21	0.02
2	88.28%	97.38%	97.36%	97.54%	30 / 39	0.34
3	88.28%	97.28%	97.52%	97.38%	33 / 26	0.43
4	92.58%	96.76%	97.08%	97.10%	40 / 41	1.00
All 25,000	88.91% †	96.90%	97.30%	97.25%	171 / 159	0.54

† mean over the 5 × 256 checked reviews.

Qwen3.8-27B reaches 97.30% OOF (95% CI 97.09–97.49%), 0.05 points above Mistral-24B. It wins in two of the five folds; over all reviews it is right on 171 that Mistral-24B gets wrong, against 159 the other way (p = 0.54, not significant). It is significantly better than Qwen3-14B (156 against 120, p = 0.035), Qwen3-8B (p = 3.2e-4) and DeBERTa (p = 6.8e-11). Its errors are nearly balanced (306 false positives, 370 false negatives). Zero-shot it starts lower than the base models (Section 4.13), but after half an epoch it is at 96.90% on average, as high as the others. It is the best of the five models on the longest reviews (98.24% on reviews of 500 words or more, Figure 14) and on reviews under 100 words (97.00%).

A new architecture did not make the errors much more different. Its margins correlate with Qwen3-14B at 0.993 and with Mistral-24B at 0.987; the Mistral-24B/Qwen3-14B pair was at 0.988. Qwen3.8-27B and Mistral-24B are both wrong on 517 reviews and agree on 98.68% of all OOF reviews.

Table 13. Combinations of the five families on the 25,000 OOF rows (estimate), paired with Qwen3.8-27B alone.

Combination	Estimate	Only combination / only 27B right	McNemar p	Role
Qwen3.8-27B alone	97.296%	–	–	
s-mean	97.344%	115 / 103	0.46	rule choice → file E
stack (logistic regression)	97.392%	–	–	gain over s-mean < 1 SE
z-mean (comparison only)	97.366%	124 / 106	0.26	
mean of raw margins (comparison only)	97.340%	114 / 103	0.50	
rank-mean (comparison only)	97.328%	108 / 100	0.63	

The rule chose s-mean (97.344%). The stack was 0.048 points higher, less than the 0.10-point standard error, so the rule did not use it. Against Qwen3.8-27B alone, s-mean fixes 115 errors and breaks 103 correct answers (p = 0.46). Against D on the OOF rows, E is right on 30 reviews where D is wrong and wrong on 20 where D is right (p = 0.20, not significant). Its estimate is 0.04 points above D, below the 0.08-point margin, so the pre-registered rule kept D. On the test set E and D differ on only 28 reviews.

Stage 7 therefore repeats the pattern of Stages 5 and 6 more clearly. The new model is at least as good as the best previous one, but it adds almost nothing to the average because it makes the same errors.

6.11 Kaggle submissions and the final selection

Table 14. Every submitted file: validation estimate and public-leaderboard accuracy (≈ 10,000 test reviews). EM = exact-match backfill.

Stage	Submission (folder)	Validation estimate	Public
1	NB-LR, uni/bi/trigrams (results)	91.78% (5-fold CV)	0.91730
2	DeBERTa-v3-large, seed 42 (runs/deberta)	96.72% (hold-out)	0.96880
2	Mean of DeBERTa and NB-LR (ensemble)	96.82% (CV in hold-out)	0.96910
2	Mean of 4 transformer runs and NB-LR (ensemble2)	96.90% (CV in hold-out)	0.97050
3	DeBERTa, mean of 5 fold models (ensemble-deberta)	96.73% (OOF)	0.97150
3	ELECTRA-large, mean of 5 fold models (ensemble-electra)	96.41% (OOF)	0.96940
3	DeBERTa fresh re-run, 5 fold models + EM	96.77% (OOF)	0.97010
4	Pilot: DeBERTa 5-fold + Qwen3 fold 0, s-mean	96.72% (fold 0)	0.97320
4	Qwen3, mean of 5 fold models (ensemble-qwen3) §	97.02% (OOF)	0.97360
4	A: Qwen3 + EM (ensemble-stage4)	97.02% (OOF) + EM	0.97380
4	B: stack of DeBERTa and Qwen3 + EM (ensemble-stage4-stack)	97.07% (OOF, CV) + EM	0.97450
5	C: s-mean of DeBERTa, Qwen3-8B and Qwen3-14B + EM (ensemble-stage5)	97.20% (OOF) + EM	0.97560
6	D: s-mean of DeBERTa, Qwen3-8B, Qwen3-14B and Mistral-24B + EM (ensemble-stage6)	97.30% (OOF) + EM	0.97620 ✓
7	E: s-mean of the four families and Qwen3.8-27B + EM (ensemble-stage7)	97.34% (OOF) + EM	0.97650 ✓

✓ selected for the private leaderboard. § uploaded twice — the Stage 4 rule produced the identical file — with the same score.

All files were checked for the header id,sentiment, for all 25,000 test ids exactly once in the sample_submission order, and for only the two allowed labels. They were uploaded under Kaggle username [removed] (account name [removed]; leaderboard team name [removed]). The Stage 1 file (SHA-256 6cd29733998ab0d2…) scored within 0.05 points of its CV estimate, and the Stage 2 DeBERTa file inside its hold-out confidence interval.

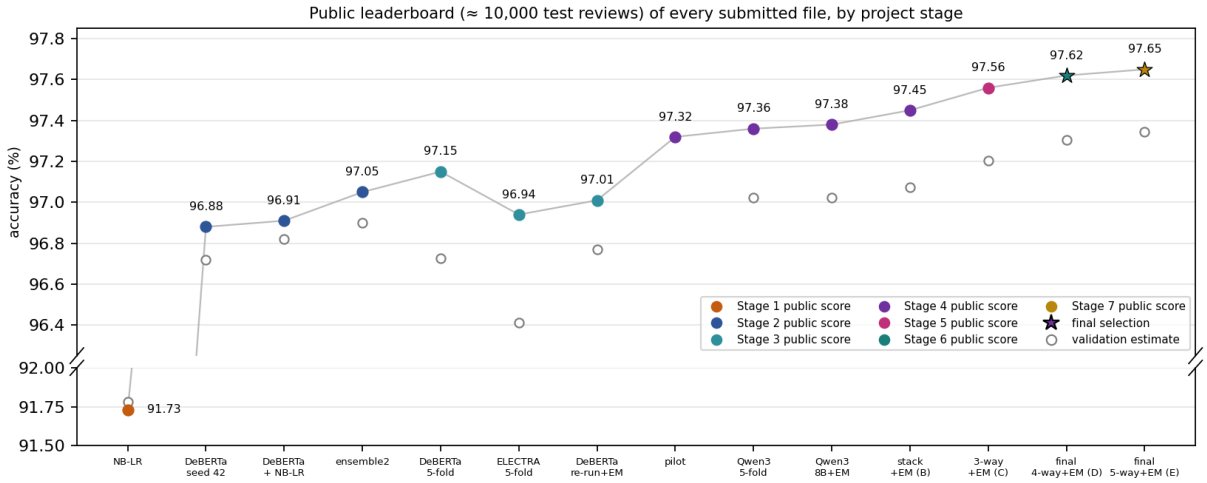


Figure 15. Public score (filled) and validation estimate (open circles) of every submitted file, grouped by stage; stars mark the two final selections (D and E).

Validation and leaderboard agree. Every stage raised both the validation estimate and the public score: 0.91730 → 0.96880 → 0.97150 → 0.97380 / 0.97450 → 0.97560 → 0.97620 → 0.97650. The 5-fold averages score 0.24–0.53 points above their OOF estimates (Qwen3: 97.02% OOF, 0.97360 public). That is more than the 0.16 points by which the single Stage 2 model beat its hold-out, and it is the expected benefit of averaging five fold models that together saw all labels, which the single-model OOF estimate leaves out. Differences between public scores are much noisier than they look, however. Qwen3 and DeBERTa disagree on 375 test reviews, and Qwen3 leads by 0.21 public points. That is in line with its 0.30-point OOF lead, but on its own only about 1.7 standard errors (Section 7.4).

The backfill. Replacing the predictions for the 227 duplicated test reviews changed 4 labels and added 0.02 public points (two reviews) to Qwen3. The same four reviews changed in the stack and in the Stage 5, 6 and 7 averages. They are hard cases. Two read as the opposite of their label (42721, labelled positive: "THE JIST: See something else … I suggest you don't accept the challenge"), and two are mixed (38371, labelled negative: "I kind of like JAG … I still watch the show, so it's not that bad"). Each transformer family had predicted against the training label on 2–4 of these reviews; NB-LR, which is refitted on all 25,000 reviews with trigram features, reproduces its training labels and needs no backfill.

How the public leaderboard was used. Stage 1 used no leaderboard feedback (Section 4.3). From Stage 2 on, public scores served as an independent check of each validation estimate. In Stage 3 they also showed that averaging the fold models helps, which the OOF estimate cannot measure. They were not used to choose between Stage 4–7 variants: the combination rules and the Stage 5–7 replacement thresholds were fixed before any result of the stage. The one selection made after a public score was known is the last one, described below.

Final selection. Kaggle ranks a participant by the better of two selected submissions on the private 60% of the test set (about 15,000 reviews). I selected:

- **D. s-mean of DeBERTa, Qwen3-8B, Qwen3-14B and Mistral-24B + backfill** (ensemble-stage6/submission_exact.csv, public 0.97620): the Stage 6 rule choice. It replaced the Stage 5 choice C (public 0.97560), which had replaced the Stage 4 choice A (public 0.97380), each time because the OOF estimate passed a threshold fixed before the run (Sections 6.8 and 6.9). The Stage 7 rule kept it, because E did not pass its threshold (Section 6.10).

- **E. s-mean of the five families + backfill** (ensemble-stage7/submission_exact.csv, public 0.97650): the Stage 7 combination. It takes the place of B, the Stage 4 stack of DeBERTa and Qwen3-8B (public 0.97450), which had been the second selection since Stage 4.

The second change was my decision, not the outcome of a pre-registered rule, and it was made after E's public score was known. The rules of Stages 5–7 only governed the slot that D now holds; nothing was fixed in advance for B's slot. The case for E over B rests on the OOF rows, not on the leaderboard. E is right on 111 OOF reviews where the cross-fitted stack B is wrong and wrong on 45 where B is right ($p = 1.3e-7$); its estimate is 0.27 points higher. The public scores point the same way (0.20 points, about 20 reviews of roughly 44 public disagreements), but on their own they would not justify the change (Section 7.4).

The cost of the change is the hedge. B differs from D on 91 test reviews, E on only 28. Kaggle counts the better of the two selections, so D + E is close to a single bet on the language-model average. It wins if that average is better than the stack on the private reviews, which the OOF comparison supports with $p = 1.2e-6$ for D and $p = 1.3e-7$ for E. Keeping B would only have helped if B were better on the private part, against this evidence. D and E are statistically indistinguishable (Section 6.10). With about 15,000 private reviews, I expect both to score about 97.2–97.9% there and to lie within a few reviews of each other.

7. Discussion

7.1 Why the errors happen

Reading the highest-confidence out-of-fold mistakes of NB-LR (listed in results/cv_summary.json) shows three recurring causes, none of which a bag-of-n-grams model can fix on its own:

- **Sarcasm and "so bad it's good".** Review 859 (true negative, predicted positive, score +8.0) opens with "This has to be one of the all time greatest horror movies ... this little seen gem. Highly realistic and incredibly stylised" and only reveals its irony through context. Review 20591 (true positive, predicted negative) praises a film precisely for its "horrible acting" being "too funny". Every surface phrase points the wrong way.
- **Mixed or comparative content.** Review 15544 (true positive, score −10.2) spends most of its words mocking the premise of 2001: A Space Odyssey before a brief positive verdict; long negative digressions outvote a short conclusion because every n-gram has equal say regardless of position.
- **Label noise.** Review 13950 ("I found Strange Fruit to be an excellent movie ... a bit rough around the edges") is labelled negative but reads as clearly positive; such cases put a hard ceiling slightly below 100% on any model and are a known property of the IMDB dataset.

7.2 What the transformer fixes, and what remains

DeBERTa corrects 158 of NB-LR's 206 hold-out errors (77%). Reading the corrections with the most confident NB-LR scores shows the patterns predicted in Section 4.4:

- **Concession followed by a verdict.** Review 22913 (negative) opens "Certainly NOMAD has some of the best horse riding scenes, swordplay, and scrumptious landscape cinematography ... but this isn't what makes a film good". NB-LR counts the praise and predicts positive; DeBERTa reads the "but" and the closing "I can't give this film a positive rating". Review 16976 (positive) is the mirror image: "My main problem with the film is that it goes on too long. Other than that, it's pretty good."
- **Complaints inside a positive review.** Review 22618 (positive) praises the acting and says the family "couldn't wait for each episode", then spends half its words on a confusing ending ("we felt

cheated"). Review 3595 (positive) calls the film "neither original nor exciting" and "mediocre" but ends "7/10" — the transformer weighs the final verdict more heavily.

The transformer's own errors are different in kind. Of its eight most confident hold-out mistakes, four read as the opposite of their label: review 7445 (labelled negative) calls the film "one of Renny Harlin's best" with Viggo Mortensen "excellent as usual"; review 4199 (labelled positive) concludes "the movie was bad". IMDB labels come from the reviewer's star rating (≤ 4 negative, ≥ 7 positive; Maas et al., 2011), and the text does not always agree with the stars. Three are "so bad it's good" or deliberately mixed reviews (review 4819: "an atrociously dumb movie, and yet … I can't help liking it", graded "D-" and "A-" in the same sentence; review 11560: "despite just how unwatchable this film really is, it is a success"), and one is ironic praise written as a rave (16785: "it surely defines the word 'masterpiece'", labelled negative). Even a human reader would struggle with most of these, which suggests that accuracy of about 97% is close to the ceiling for this dataset. Section 7.3 tests this suggestion with the Stage 4 model.

7.3 What the language model fixes, and what remains

Reading the reviews on which Qwen3 and DeBERTa disagree most confidently (listed in results/stage34/stage34_summary.json) shows what the language model adds:

- **Sarcasm built from positive words.** Review 14548 (negative) calls Flatliners "the ideal showcase for Joel Schumacher's glorious, flamboyant, brazen lack of talent" and goes on: "I love how the med school autopsy room is a cavernous vaulted marble mausoleum …". DeBERTa reads the enthusiasm (margin +4.3), Qwen3 the irony (−7.2).
- **Praise or criticism aimed at something else.** Review 2802 (negative) praises "the original masterpiece", an earlier film, and accuses the reviewed remake of plagiarism (DeBERTa +5.0, Qwen3 −5.5). Review 8390 (positive) attacks another user's "pathetically composed comments" while defending the film (DeBERTa −6.0, Qwen3 +4.7).
- **A verdict that overturns the opening.** Review 3449 (negative) opens "This was a very good show" and ends "Great performances by a fine cast cannot redeem this lightweight screenplay" (DeBERTa +3.4, Qwen3 −6.9). Review 20119 (negative) sounds positive except for its rating notation "Plot: *1/2 out of ****" (DeBERTa +5.3, Qwen3 −7.6).

Conversely, several of DeBERTa's wins are reviews whose label follows a stated score that contradicts their tone. Review 467 (negative) is warm throughout and ends "why I'm giving this a 4". Review 9876 (positive) is a rant that begins "I gave it a seven only because the acting is good".

What remains is largely label noise. Both models are wrong on 520 training reviews (2.1%). I read the 15 on which both are most confident:

- 11 read as the opposite of their label. For example, review 15183 (labelled positive) ends "Halloween 6 only gets 4 out of 10 stars from me", and review 20942 (labelled positive) ends "Don't see this film.. you'll probably be disappointed!".
- 2 are "so bad it's good" reviews by authors who enjoyed a bad film.
- 2 are sustained mock praise (review 4919: "You can cut the sexual tension with a dull tire iron").

A human check of the original IMDB test set judged 725 of its 25,000 labels (2.9%) to be wrong (Northcutt et al., 2021). That is the same order as the 2.1% of reviews that both of our best models still get wrong, and the backfilled reviews of Section 6.11 are of the same kind. Gains of more than a few tenths of a point are therefore unlikely for any model trained and scored on these labels.

7.4 How noisy is the public leaderboard?

The public score is computed on about 10,000 test reviews, so a single score has a standard error of about 0.16 points ($\sqrt{(0.973 \times 0.027 / 10{,}000)}$). When two submissions are compared, what matters is the number d of public reviews on which they disagree: if neither file is truly better, the net difference in correct reviews has a standard deviation of about $\sqrt{d}$. Six comparisons from this project (taking 40% of each file pair's test disagreements to be public):

- Two runs of the same DeBERTa recipe (Section 6.5) disagree on 110 test reviews, about 44 of them public. Their 14-review (0.14-point) difference is two standard deviations, although their OOF accuracies are statistically identical.
- Qwen3 against DeBERTa (5-fold averages): about 150 public disagreements; the difference of 21 reviews is about 1.7 standard deviations, consistent with the significant OOF gain.
- B against A (Stage 4): about 48 public disagreements; the difference of 7 reviews is one standard deviation.
- C against B (Stage 5): about 29 public disagreements; the difference of 11 reviews is two standard deviations, consistent with the significant OOF gain.
- D against C (Stage 6): about 17 public disagreements; the difference of 6 reviews is 1.5 standard deviations, in the direction of the significant OOF gain but not decisive on its own.
- E against D (Stage 7): about 11 public disagreements; the difference of 3 reviews is under one standard deviation, matching the non-significant OOF difference.

A public lead of a few hundredths of a point between similar files is therefore no evidence at all. Choosing the best of many variants by public score would mostly select noise, which does not carry over to the private part. This is why every decision in Stages 3–7 was based on the 25,000 OOF rows, which contain 2.5 times as many reviews and where each comparison is made review by review.

7.5 Strengths and limitations of the approach

NB-LR (Stage 1): fast (seconds to train on a CPU), fully reproducible bit for bit, and interpretable — every prediction is a sum of readable phrase weights. Its ≈ 91.8% accuracy matches the published figure for NB-SVM with trigrams on the original IMDB split (91.2–91.9%; Wang & Manning, 2012; Mesnil et al., 2015), which validates the from-scratch implementation. Its limitations are structural: word order beyond three tokens is lost, so contrast, long-range negation and discourse structure are invisible, and the learning curve shows it is still data-limited at 25,000 reviews.

Fine-tuned transformers (Stage 2): five points more accurate and balanced between the classes, because pre-training supplies linguistic knowledge that 25,000 reviews could never teach. The costs are a GPU (6–21 minutes per run here, but many hours on a CPU), 1.4–1.7 GB per model, predictions that cannot be explained phrase by phrase, and only approximate reproducibility when re-training. It also truncates the longest 12% of reviews, and in Stage 2 it was evaluated on one 2,500-review hold-out rather than by 5-fold CV, so its estimate was less precise (±0.7 points) than the Stage 1 estimates; Stage 3 removed this limitation. Section 6.4 also showed that a single run depends noticeably on its random seed (±0.3 points), so one run is a less reliable basis for conclusions than one CV estimate in Stage 1; averaging several runs reduces this dependence but adds little accuracy, because the runs make largely the same errors.

Cross-validation (Stage 3) made every later decision measurable: with 25,000 OOF rows a 0.3-point difference becomes significant. It cost five times the GPU time of a single run, and selecting DeBERTa's

checkpoints on the validation folds made its OOF estimate slightly optimistic (by about 0.08 points). The averaged fold models also scored higher on the leaderboard than any Stage 2 file.

The LoRA-adapted language model (Stage 4) is the most accurate single model of Stages 1–4 and the main reason the final score rose by a further 0.2–0.3 points. It is also by far the most expensive: 5.8 GPU-hours for five folds on a 24 GB GPU, a 16 GB download, and 19 minutes for each fold model to predict the test set, against under 2 minutes for DeBERTa. It has further limitations. Its hyper-parameters were fixed, not tuned, and only one seed was run. IMDB pages in its pre-training data cannot be ruled out (Section 4.8). Its errors overlap strongly with the encoders' (Section 6.6), which limits what combining them can add. Finally, the stack weights are fitted to single fold-model margins but applied to averages of five fold models, whose spread is slightly smaller; with fold-model correlations above 0.99 this has little effect (the stack predicts 50.06% positive on the test set, against 49.96% for Qwen3).

The larger language model (Stage 5) answered whether scale still helps: it does, but slowly. Moving from 8 to 14 billion parameters gained 0.13 points, not significant on its own, at the price of renting a 96 GB GPU (4.6 GPU-hours for the folds, about ¥37 for the whole rental). Its main value was as a second strong vote: the three-model average is the first combination in this project that is significantly better, on the OOF rows, than both Stage 4 files. Both Qwen3 models share most of their errors (581 of them), so further scaling within one model family is unlikely to pay off.

The differently pre-trained model (Stage 6) tested the remedy suggested by Stage 5: a model from another developer, trained on other data. It is the best single model (97.25% OOF) and its margins are a little less correlated with Qwen3-14B (0.988) than the two Qwen3 models with each other (0.992), but its errors still overlap heavily, and the four-model average is not significantly better than Mistral-24B alone. It needed an 80 GB GPU for 7.4 GPU-hours, including one fold retrained after an unstable run (Section 6.9). Three language models from two developers now agree on most of their errors; this supports the conclusion that the remaining errors come largely from the labels.

The newer architecture (Stage 7) gave the same answer. Qwen3.8-27B, with linear attention in three quarters of its layers, matched Mistral-24B (97.30% OOF) in 11.4 GPU-hours on a 96 GB GPU and needed a newer software stack. Its errors overlap with the other language models as much as theirs overlap with each other, and the five-model average is not significantly better than the four-model average. Because no base checkpoint exists, it is also the only model in the project whose post-training data might include sentiment labels (Section 4.13), so its OOF accuracy is a less clean measure of what fine-tuning achieves than the others'.

Together, the seven stages show that each expensive step was justified by the analysis of the previous one. The error analysis of the cheap model predicted where the transformer would help (Sections 7.1–7.2). The noise of the small hold-out motivated cross-validation (Section 6.4). The near-identical encoders motivated a differently pre-trained model (Section 4.8). The modest Stage 4 gain left the question of scale, which Stage 5 answered with a measured, pre-registered test, and the shared errors of the two Qwen3 models motivated a model from another developer in Stage 6 and a different architecture in Stage 7. And the reading of the remaining errors (Section 7.3) suggests that the labels, not the models, now set the limit.

7.6 What I would try next

Three of the four suggestions in the Stage 2 version of this report were carried out: using all 25,000 labels (Stage 3), a model with a different pre-training objective and a longer input limit (Stage 4), and reading almost every review whole (1,024 tokens). The Stage 5 suggestion of a language model from another

developer was carried out in Stage 6, and Stage 7 added a different architecture. Neither changed the conclusion, so a further, larger model is not on the list. What remains:

- More training for the language models: two epochs and several seeds. Accuracy was still rising between half an epoch and the end in four of five folds, and averaging seeds would reduce the variance of the single run.
- Handle label noise explicitly. Training reviews whose label contradicts every model (Section 7.3) could be flagged from the OOF margins (confident learning; Northcutt et al., 2021) and down-weighted in training. Removing them from the evaluation would make the validation numbers look better, but not the leaderboard score, because the test labels are noisy in the same way.
- Domain-adaptive pre-training (Gururangan et al., 2020) of DeBERTa on the 50,000 unlabelled train and test review texts. It uses no labels, is cheap compared with the LLM, and might make DeBERTa a stronger partner for the language models.

7.7 Use of generative AI

In line with the course policy, AI coding assistants were used as pair programmers and editors throughout. Claude (Anthropic) helped draft the NumPy implementations, the experiment drivers, the Stage 2–7 fine-tuning, cross-validation, LLM and analysis scripts, the step-by-step instructions for running them on my own GPU and on the rented cloud GPUs, and this report. OpenAI Codex helped optimise the Stage 1 code and add unit tests. The assistants proposed methods and wrote most of the code under my direction. I decided which experiments to run and which files to submit, ran every training job on my own computer or on GPUs I rented, uploaded all submissions, checked the numbers in this report against the saved result files, and I am responsible for the final content. The Stage 4–7 models are themselves generative language models, used here as a classifier, not to generate text.

8. References

[1] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Portland, Oregon, 142–150. http://www.aclweb.org/anthology/P11-1015. Dataset: https://ai.stanford.edu/~amaas/data/sentiment/

[2] Wang, S., & Manning, C. D. (2012). Baselines and bigrams: Simple, good sentiment and topic classification. Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics, 90–94.

[3] Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). Spam filtering with naive Bayes — which naive Bayes? Third Conference on Email and Anti-Spam (CEAS).

[4] Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. Proceedings of EMNLP 2002, 79–86.

[5] Mesnil, G., Mikolov, T., Ranzato, M., & Bengio, Y. (2015). Ensemble of generative and discriminative techniques for sentiment analysis of movie reviews. ICLR Workshop. arXiv:1412.5335.

[6] Liu, D. C., & Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. Mathematical Programming, 45, 503–528.

[7] Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. Foundations and Trends in Information Retrieval, 2(1–2), 1–135.

[8] Sun, C., Qiu, X., Xu, Y., & Huang, X. (2019). How to fine-tune BERT for text classification? China National Conference on Chinese Computational Linguistics, 194–206.

[9] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems 30, 5998–6008.

[10] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of NAACL-HLT 2019, 4171–4186.

[11] He, P., Gao, J., & Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. International Conference on Learning Representations (ICLR 2023). arXiv:2111.09543.

[12] He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with disentangled attention. International Conference on Learning Representations (ICLR 2021). arXiv:2006.03654.

[13] Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). ELECTRA: Pre-training text encoders as discriminators rather than generators. International Conference on Learning Representations (ICLR 2020).

[14] Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. Proceedings of EMNLP 2018: System Demonstrations, 66–71.

[15] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692.

[16] Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., & Smith, N. A. (2020). Don't stop pretraining: Adapt language models to domains and tasks. Proceedings of ACL 2020, 8342–8360.

[17] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. International Conference on Learning Representations (ICLR 2019).

[18] Wolf, T., et al. (2020). Transformers: State-of-the-art natural language processing. Proceedings of EMNLP 2020: System Demonstrations, 38–45.

[19] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. (Used only as the reference for the TF-IDF and logistic-regression parameterisations; the library itself was not used.)

[20] Yang, A., et al. (Qwen Team) (2025). Qwen3 technical report. arXiv:2505.09388. Models used: Qwen/Qwen3-8B-Base and Qwen/Qwen3-14B-Base, Apache-2.0 licence.

[21] Mistral AI (2025). Mistral-Small-24B-Base-2501 model card. Hugging Face. Model used: mistralai/Mistral-Small-24B-Base-2501, Apache-2.0 licence. https://huggingface.co/mistralai/Mistral-Small-24B-Base-2501

[22] Qwen Team (2026). Qwen3.8-27B model card. Hugging Face / ModelScope. Model used: Qwen/Qwen3.8-27B (post-trained, Apache-2.0 licence). https://huggingface.co/Qwen/Qwen3.8-27B

[23] Yang, S., Kautz, J., & Hatamizadeh, A. (2025). Gated Delta Networks: Improving Mamba2 with delta rule. International Conference on Learning Representations (ICLR 2025). arXiv:2412.06464.

[24] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. International Conference on Learning Representations (ICLR 2022). arXiv:2106.09685.

[25] Northcutt, C. G., Athalye, A., & Mueller, J. (2021). Pervasive label errors in test sets destabilize machine learning benchmarks. NeurIPS 2021 Track on Datasets and Benchmarks. arXiv:2103.14749.

[26] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), 153–157.

[27] Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241–259.

[28] Xie, Q., Luong, M.-T., Hovy, E., & Le, Q. V. (2020). Self-training with Noisy Student improves ImageNet classification. Proceedings of CVPR 2020, 10687–10698.

[29] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. Proceedings of EMNLP 2023, 4895–4901.

[30] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. Neurocomputing, 568, 127063.

[31] Shazeer, N. (2020). GLU variants improve transformer. arXiv:2002.05202.

[32] Zhang, B., & Sennrich, R. (2019). Root mean square layer normalization. Advances in Neural Information Processing Systems 32.

[33] Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., & Bossan, B. (2022). PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft (software; version 0.18.1 in Stages 4–6 and 0.21.2 in Stage 7; Apache-2.0).

Appendix A. Full cross-validation results

All 42 configurations, sorted by mean 5-fold CV accuracy. Produced by code/run_pipeline.py (results/cv_results.csv).

#	Configuration	n-gram	CV mean	CV std	Fit s/fold
1	NB-LR (uni+bi+tri, C=1.0, beta=0.75)	1–3	91.78%	0.35%	3.4
2	NB-LR (uni+bi+tri, C=0.3, beta=0.75)	1–3	91.74%	0.35%	3.4
3	NB-LR (uni+bi+tri, C=1.0, beta=0.5)	1–3	91.72%	0.39%	3.4
4	NB-LR (uni+bi+tri, C=0.3, beta=0.5)	1–3	91.70%	0.36%	3.4
5	NB-LR (uni+bi, C=1.0, beta=0.5)	1–2	91.66%	0.43%	2.9
6	NB-LR (uni+bi, C=0.3, beta=0.5)	1–2	91.65%	0.42%	2.7
7	NB-LR (uni+bi+tri, C=0.1, beta=0.75)	1–3	91.62%	0.28%	2.9
8	NB-LR (uni+bi, C=0.3, beta=0.75)	1–2	91.58%	0.42%	2.7
9	NB-LR (uni+bi, C=1.0, beta=0.75)	1–2	91.57%	0.34%	2.9
10	NB-LR (uni+bi+tri, C=0.1, beta=0.5)	1–3	91.52%	0.26%	2.9
11	NB-LR (uni+bi, C=0.1, beta=0.5)	1–2	91.40%	0.33%	2.3
12	NB-LR (uni+bi, C=0.1, beta=0.75)	1–2	91.40%	0.37%	2.3
13	TF-IDF + Logistic Regression (uni+bi+tri, C=16.0)	1–3	91.30%	0.37%	11.5
14	TF-IDF + Logistic Regression (uni+bi+tri, C=64.0)	1–3	91.30%	0.38%	9.0
15	TF-IDF + Logistic Regression (uni+bi, C=64.0)	1–2	91.30%	0.59%	3.4
16	TF-IDF + Logistic Regression (uni+bi, C=16.0)	1–2	91.27%	0.53%	6.4
17	NB-LR (uni+bi, C=1.0, beta=0.25)	1–2	91.20%	0.41%	2.9
18	NB-LR (uni+bi+tri, C=1.0, beta=0.25)	1–3	91.14%	0.34%	3.4
19	NB-LR (uni+bi, C=0.3, beta=0.25)	1–2	91.05%	0.39%	2.7
20	NB-LR (uni+bi+tri, C=0.3, beta=0.25)	1–3	91.04%	0.36%	3.4
21	TF-IDF + Logistic Regression (uni+bi, C=4.0)	1–2	91.00%	0.35%	4.6
22	TF-IDF + Logistic Regression (uni+bi+tri, C=4.0)	1–3	90.84%	0.38%	6.9
23	NB-LR (uni+bi, C=0.1, beta=0.25)	1–2	90.80%	0.37%	2.3
24	NB-LR (uni+bi+tri, C=0.1, beta=0.25)	1–3	90.74%	0.37%	2.9
25	TF-IDF + Logistic Regression (uni, C=4.0)	1–1	89.92%	0.30%	2.7
26	NB-LR (uni, C=0.3, beta=0.75)	1–1	89.78%	0.28%	1.6
27	TF-IDF + Logistic Regression (uni, C=16.0)	1–1	89.70%	0.40%	2.9
28	NB-LR (uni, C=0.3, beta=0.5)	1–1	89.69%	0.39%	1.5
29	NB-LR (uni, C=1.0, beta=0.5)	1–1	89.68%	0.31%	2.0
30	NB-LR (uni, C=1.0, beta=0.75)	1–1	89.56%	0.26%	2.0
31	NB-LR (uni, C=0.1, beta=0.75)	1–1	89.54%	0.31%	1.1
32	NB-LR (uni, C=0.1, beta=0.5)	1–1	89.43%	0.45%	1.1
33	NB-LR (uni, C=0.3, beta=0.25)	1–1	89.27%	0.58%	1.6
34	Multinomial NB (binary, uni+bi+tri, alpha=1.0)	1–3	89.20%	0.78%	0.1
35	TF-IDF + Logistic Regression (uni, C=64.0)	1–1	89.17%	0.35%	4.4
36	NB-LR (uni, C=1.0, beta=0.25)	1–1	89.11%	0.49%	2.0
37	NB-LR (uni, C=0.1, beta=0.25)	1–1	88.72%	0.58%	1.1
38	Multinomial NB (counts, uni+bi+tri, alpha=1.0)	1–3	88.63%	0.72%	0.1
39	Multinomial NB (binary, uni+bi, alpha=1.0)	1–2	88.11%	0.98%	0.1
40	Multinomial NB (counts, uni+bi, alpha=1.0)	1–2	87.50%	0.76%	0.1
41	Multinomial NB (binary, uni, alpha=1.0)	1–1	85.17%	0.84%	0.0
42	Multinomial NB (counts, uni, alpha=1.0)	1–1	84.06%	0.74%	0.0